

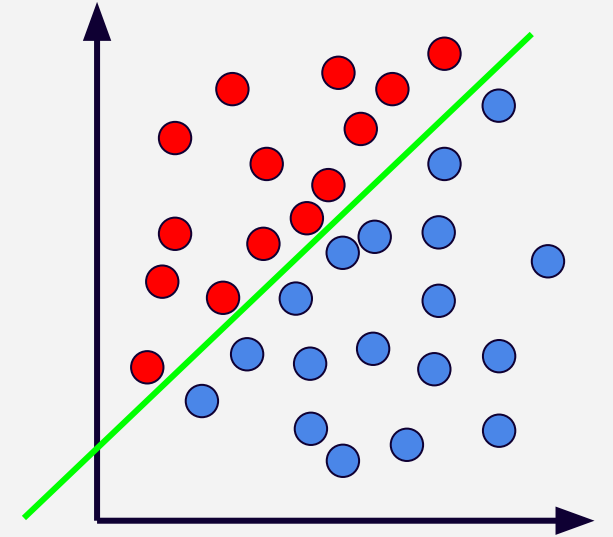
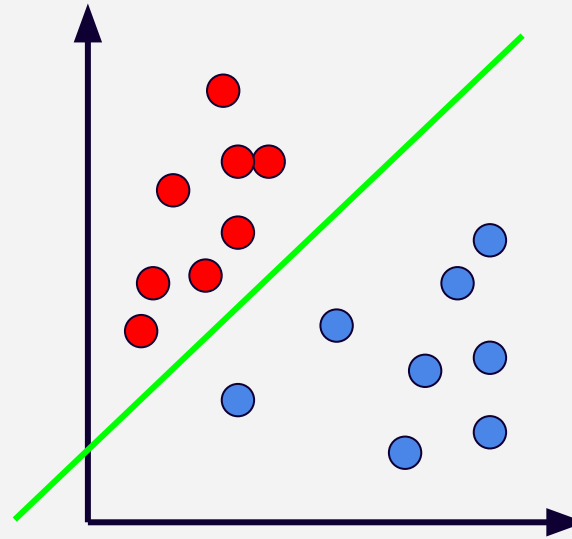
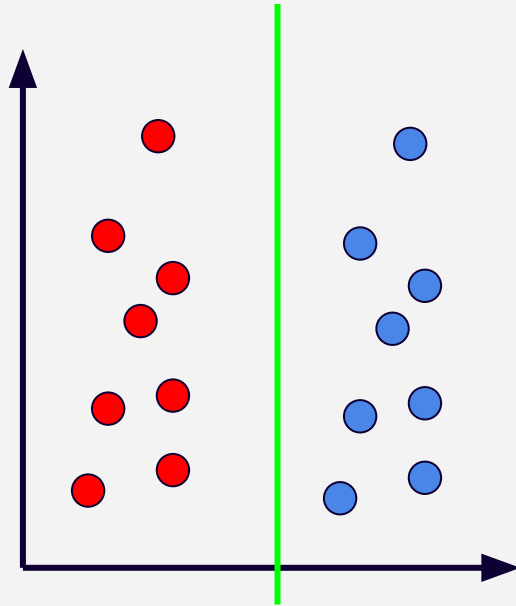
Límites de modelos lineales

Regresión Logística y Lineal Múltiples

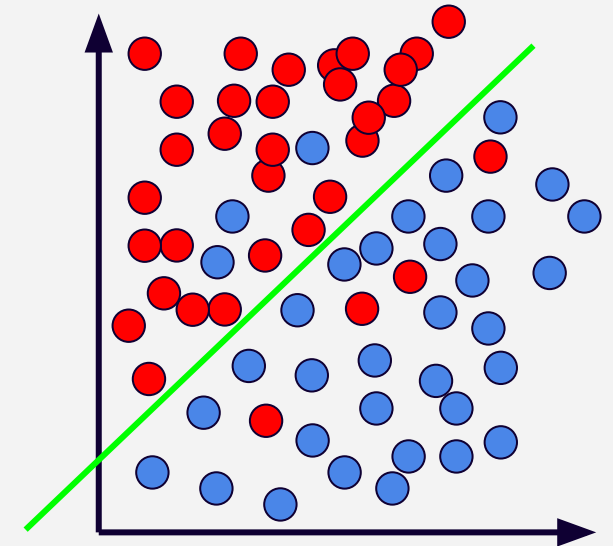
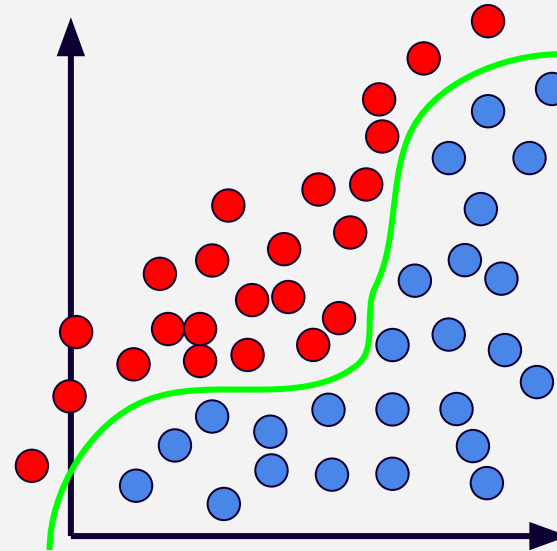
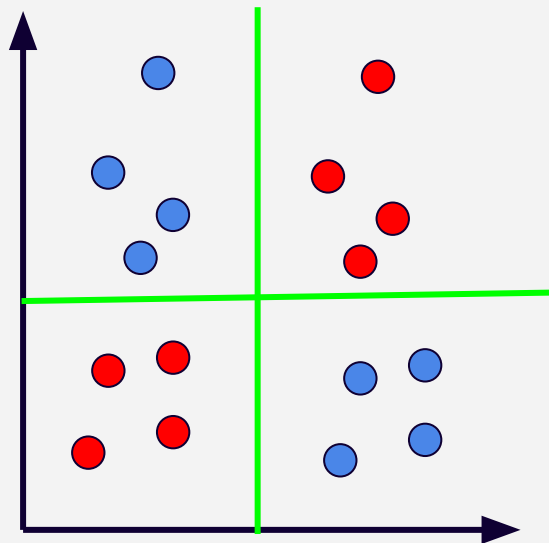
- Herramientas simples pero poderosas.
- Permiten
 - Transformaciones arbitrarias entre vectores (lineal).
 - Dar estructura de distribución de probabilidad a la salida (logística).
- Lo primero a intentar en cualquier problema
- Ventajas
 - Caja blanca, interpretable
 - Error convexo, solución única, fácil de entrenar.
- Desventajas
 - Modelos lineales
 - No pueden resolver problemas no-lineales.

Separabilidad lineal

- Linealmente Separables

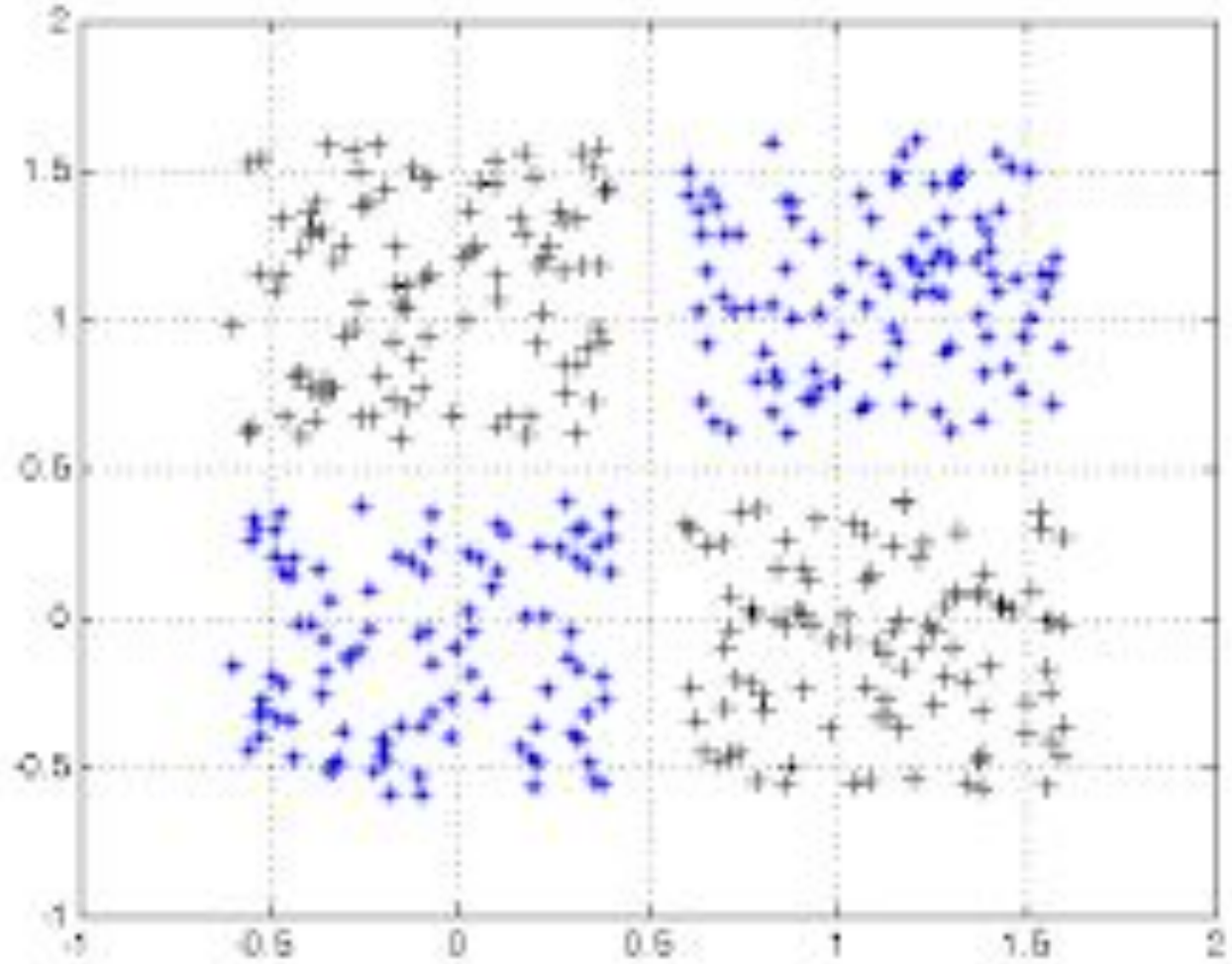


- No Linealmente Separables

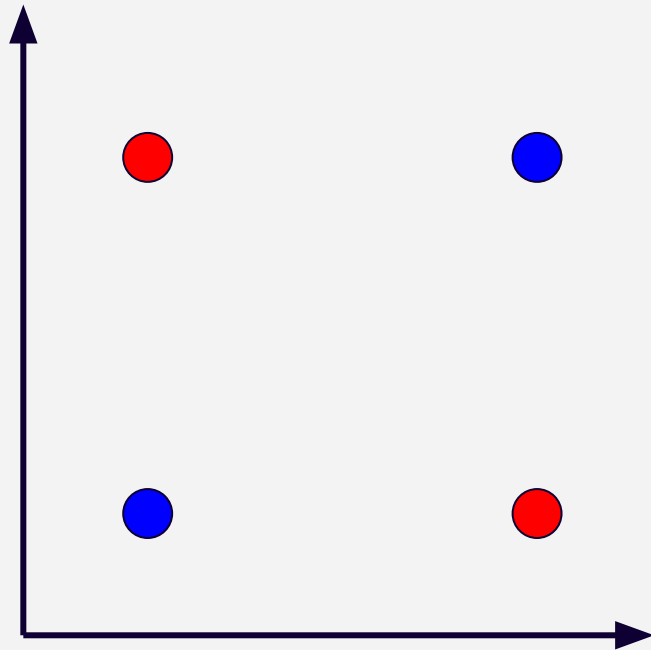


Límites de Regresión Logística - XOR

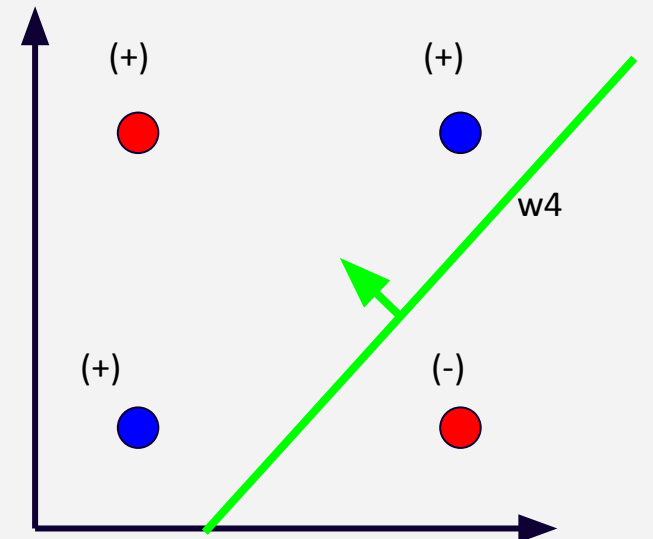
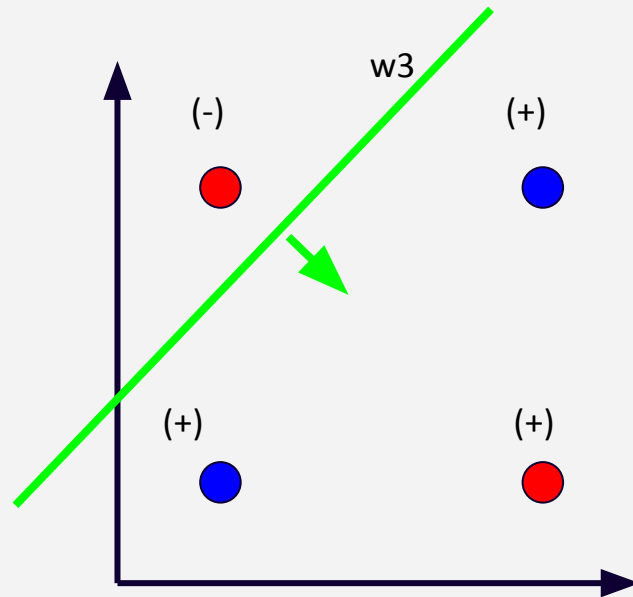
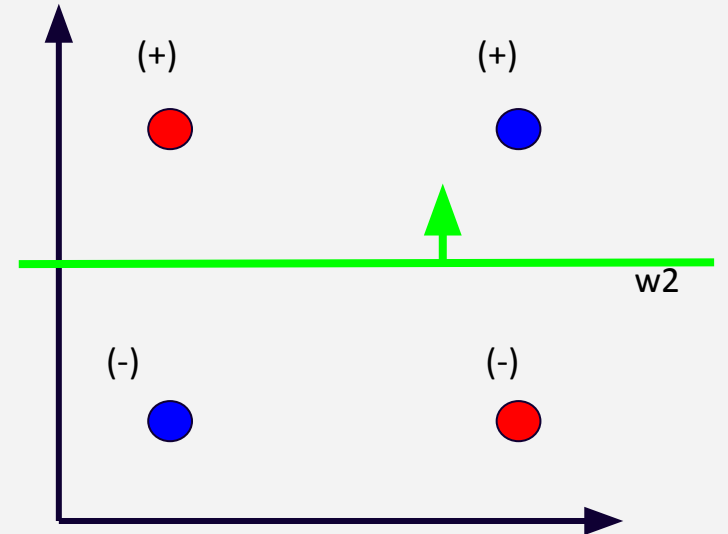
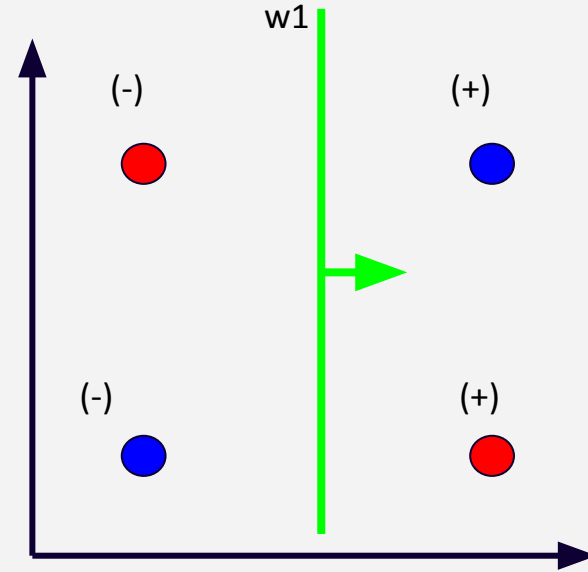
- El problema del XOR
- 2 clases
- No son **linealmente separables**
 - No pueden separarse con un solo hiperplano



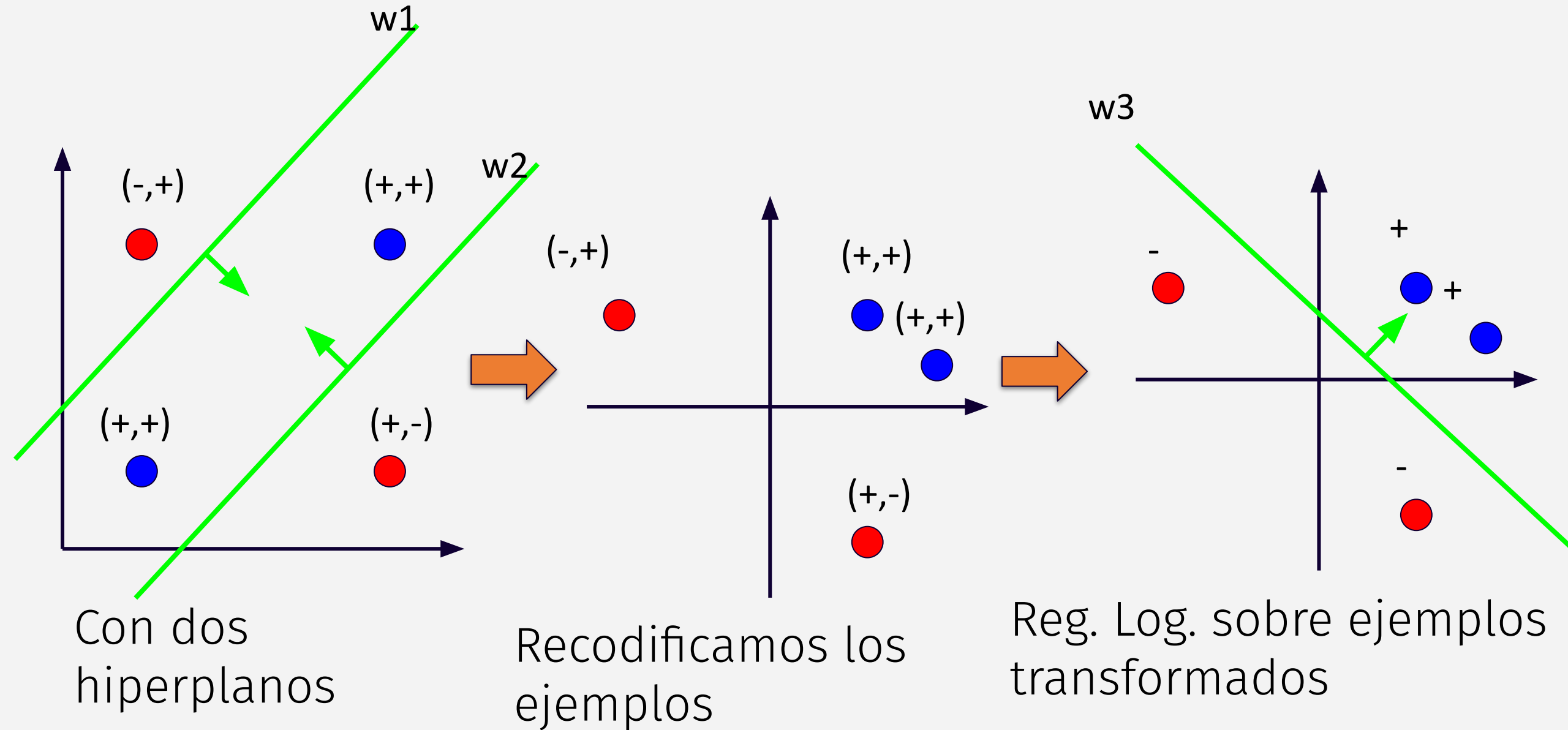
Límites de Regresión Logística - XOR



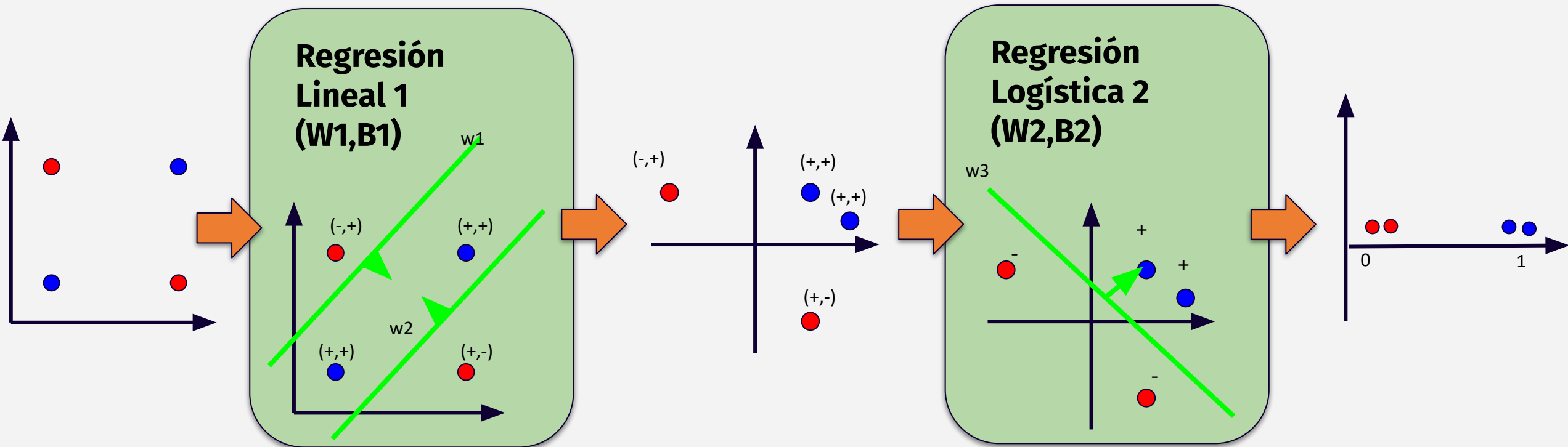
Datos originales



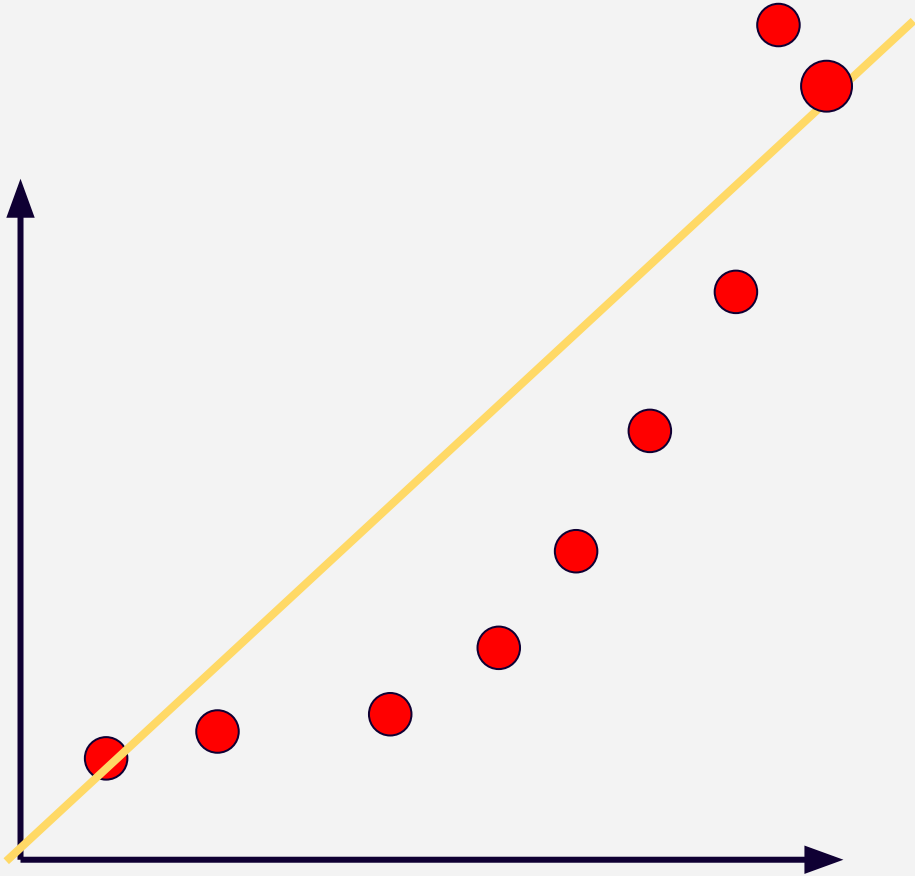
Solución con 2 Regresiones



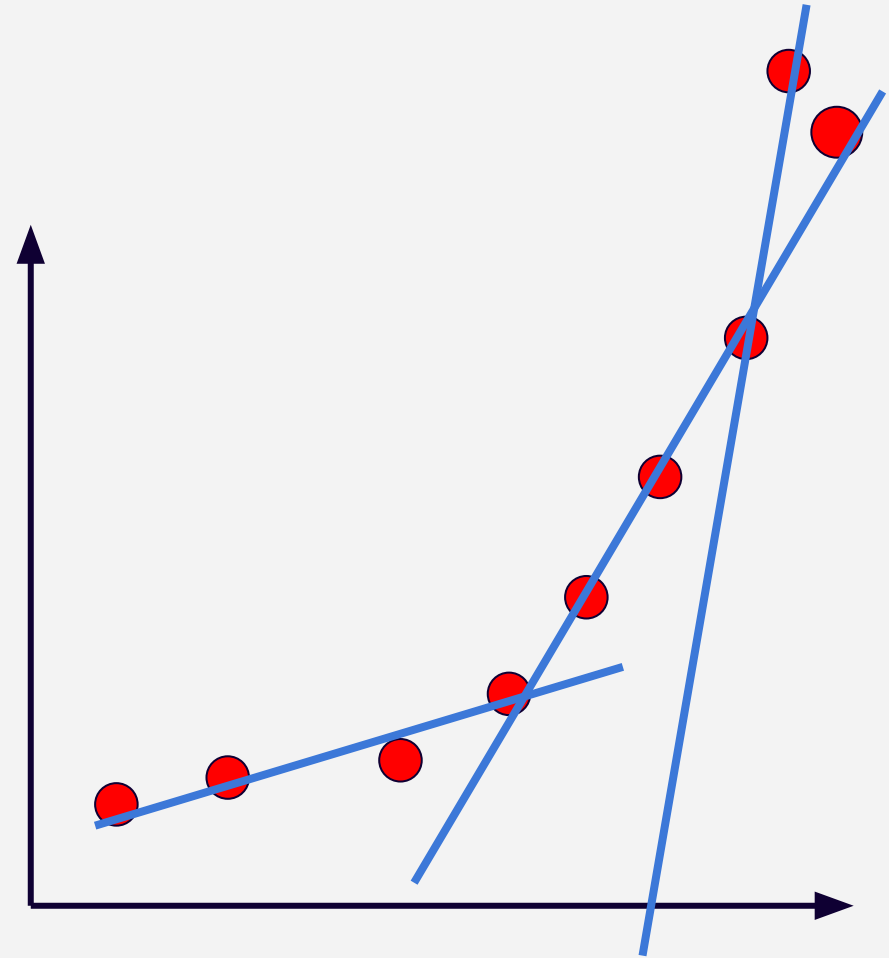
Solución con 2 Regresiones



Límites de Regresión Lineal

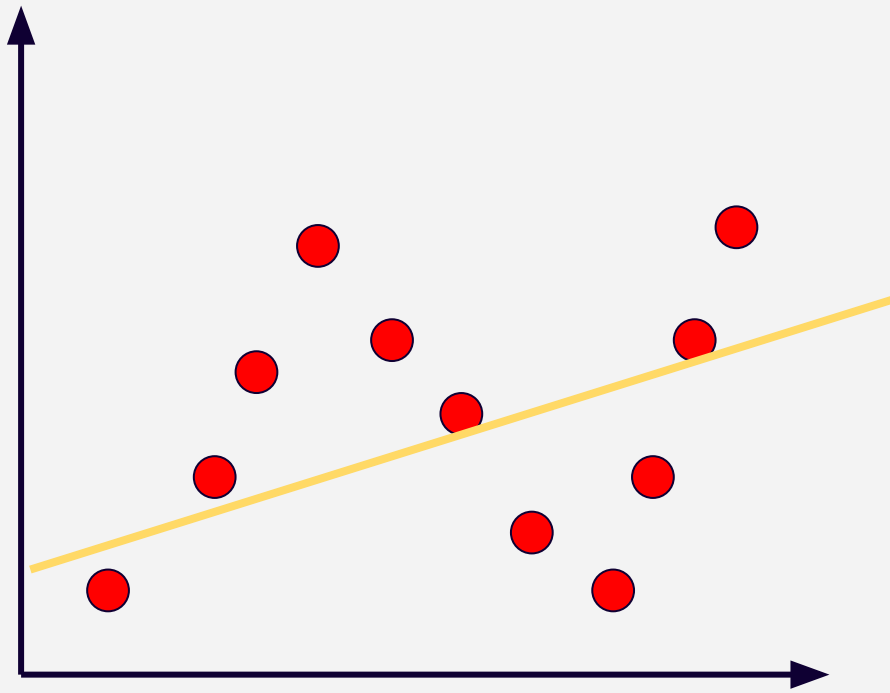


Mucho error con una
sola Regresión Lineal

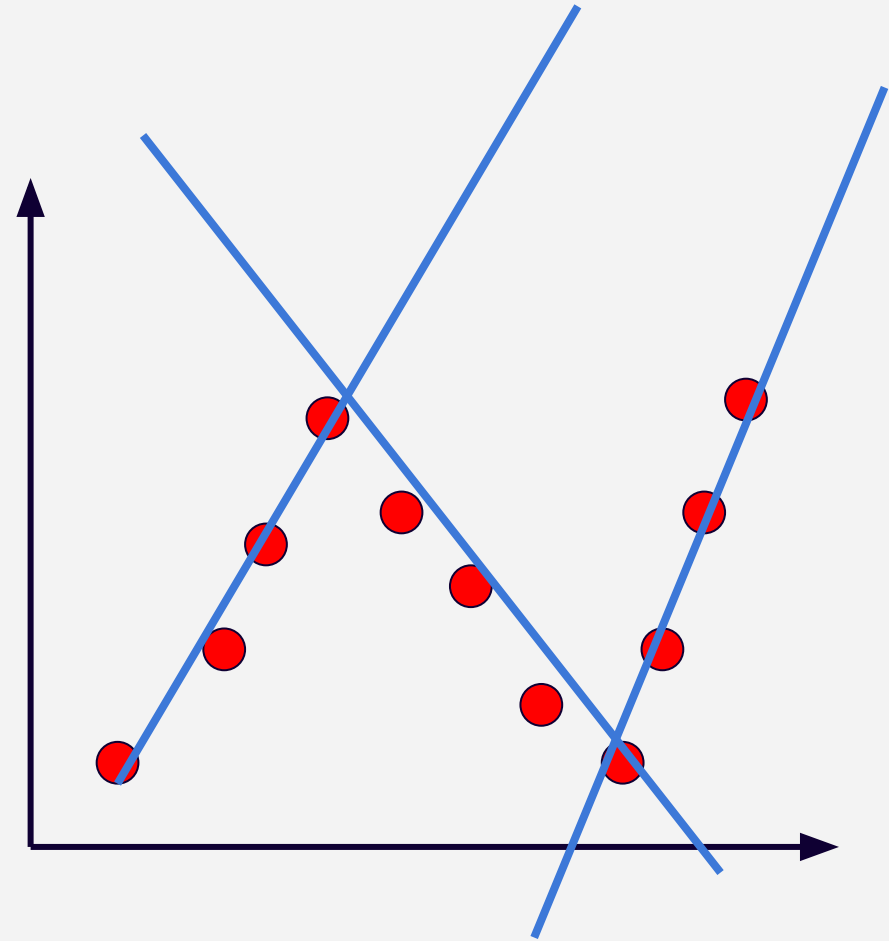


Mejora combinando
transformaciones

Límites de Regresión Lineal



Mucho error con una
sola Regresión Lineal



Mejora combinando
transformaciones

Resumen

- Regresión Lineal o Regresión Logística
 - Modelos lineales
 - Problema **linealmente separable**
- Combinar varias transformaciones
 - Aumenta el poder de los modelos
 - **Redes Neuronales**
 - Problema **no lineal(mente separable)**
 - Mayoría de los interesantes

Redes Neuronales

Aprendizaje Automático - Los Elementos

Ejemplos
(etiquetados)

Estu dio	Edad	Pro medi o	N1	N2
2	24	4	7	7.2
5	22	3	4.5	5.2
7	25	4	6.3	6

Algoritmo de
Aprendizaje

- Descenso de Gradiente

Error a optimizar

- Error cuadrático
- Entropía
- Otros

Ejemplos
nuevos

Estudi o	Edad	Prom edio
10	19	4
11	20	3

Modelo

Parámetros
(entrenados)

Predicciones

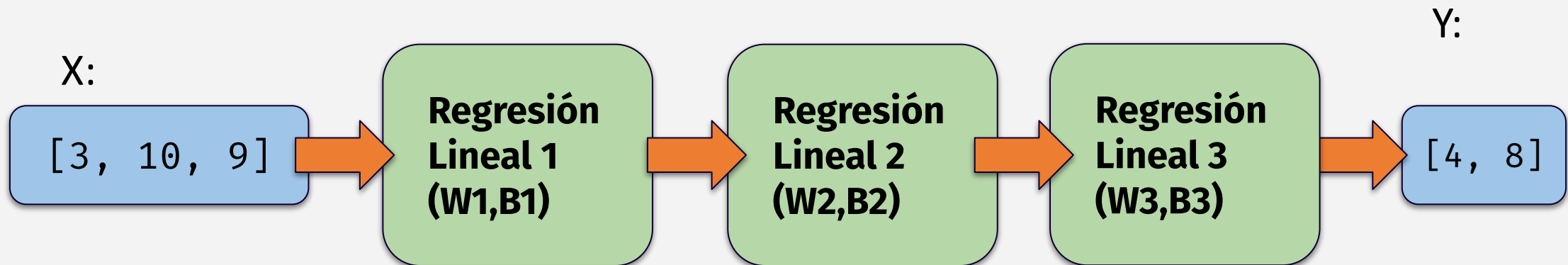
P1	P2
7	7.2
4.5	5.2

Modelo

Parámetros
(aleatorios)

Redes Neuronales

- Red neuronal para predecir notas
- Idea básica
 - Apilar transformaciones
 - Regresiones lineales
 - Otras
- Transformación = Capa
- Red de ejemplo:

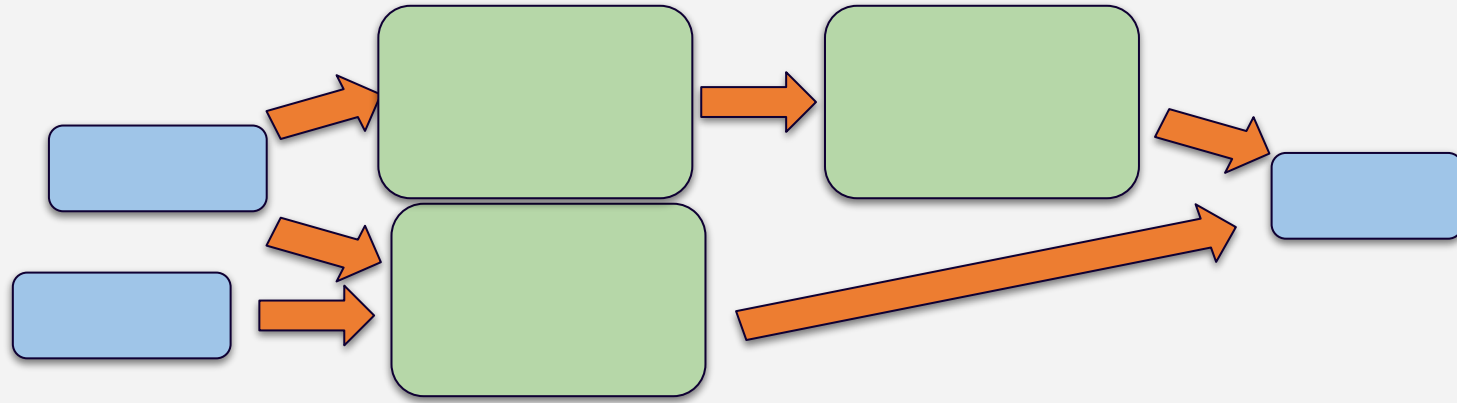


Topologías posibles de red

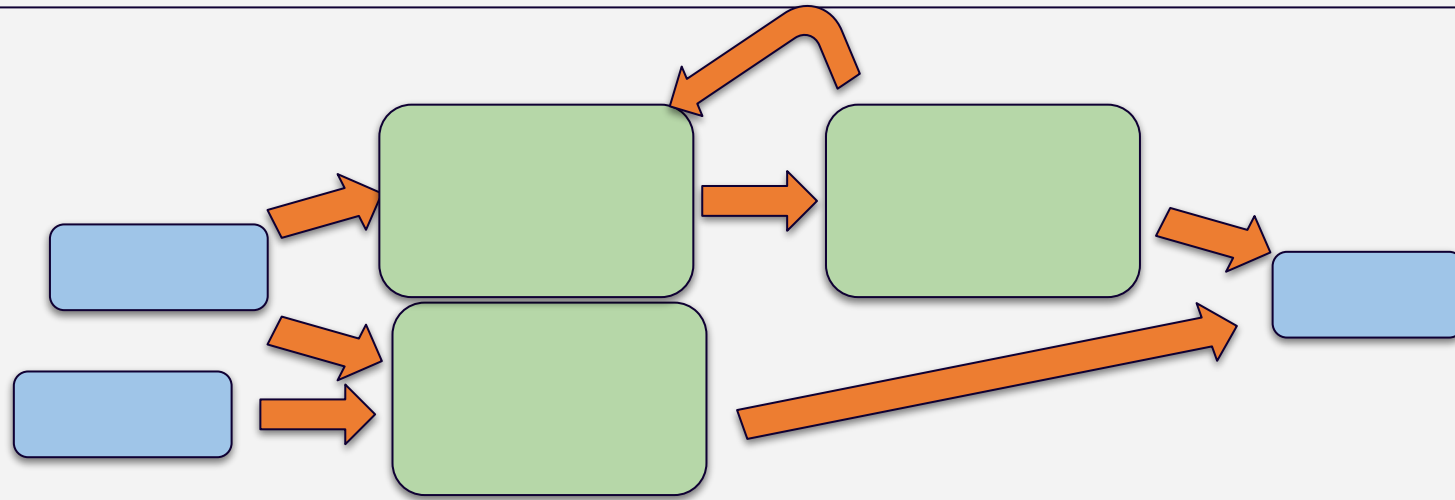
**Lineal
(Secuencial)**



Acíclico

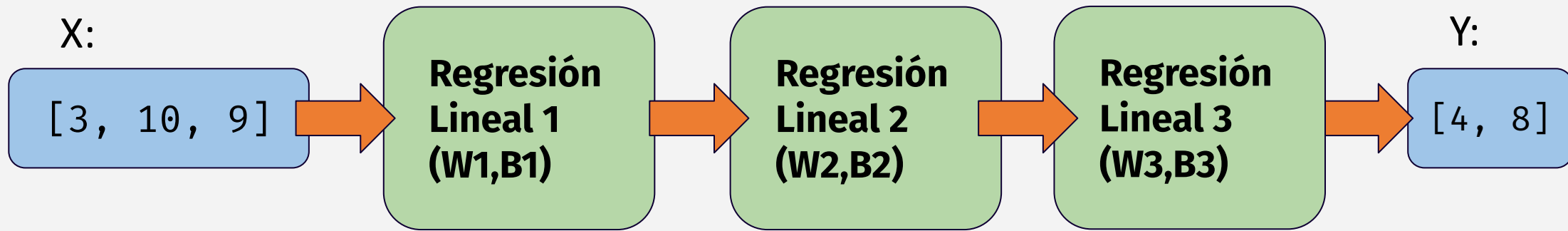


Cíclico



forward : cálculo de la salida de la red

- Salida de la red
 - Cálculo **forward**



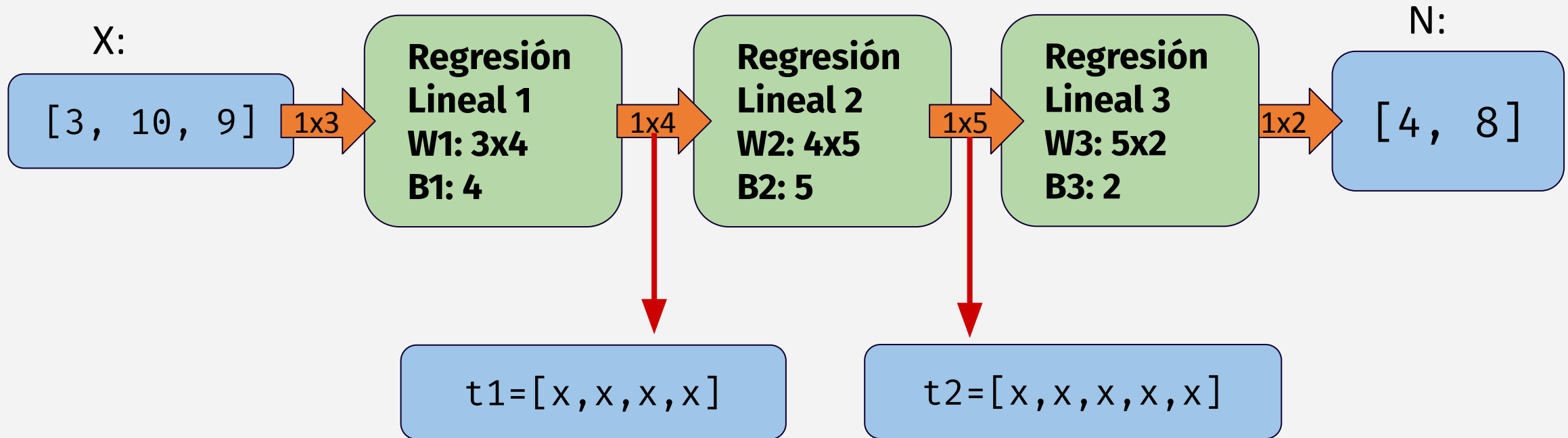
RL 1 recibe tamaño 3
=> W1 tiene tamaño 3x?

RL 2: libre de decidir
tamaños de
entrada/salida

RL 3 tiene que generar
algo de tamaño 2
=> W3 tiene tamaño ?x2
B3 tiene tamaño 2

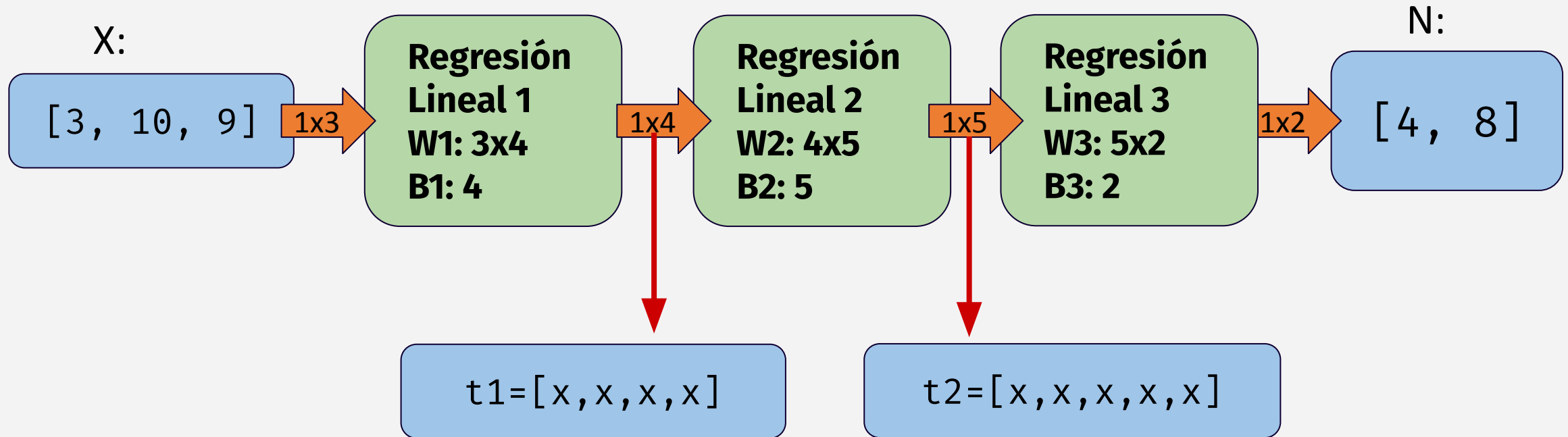
- Usamos 3 regresiones
 - Podemos usar Y
 - Depende del problema

forward y tamaño de parámetros y salidas



- $t1$ y $t2$
 - Valores intermedios
 - Se descartan

forward en detalle



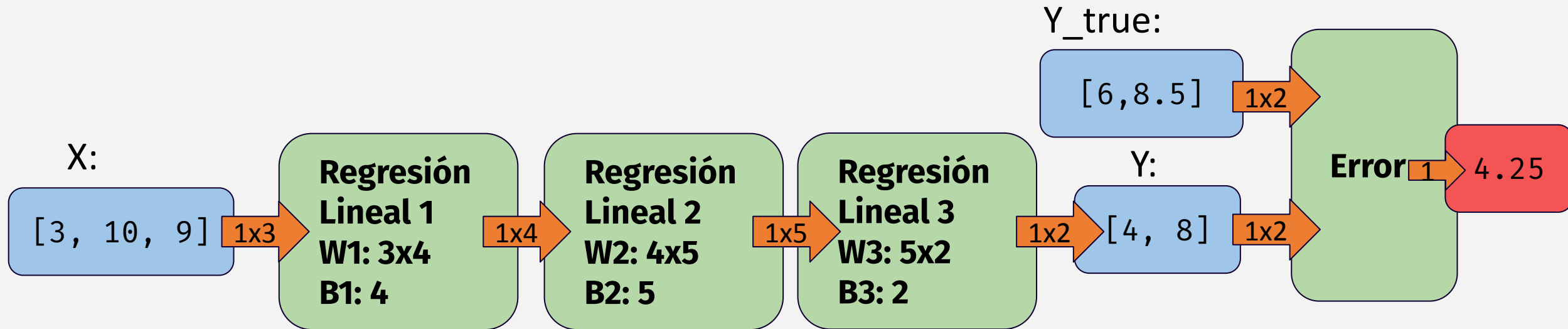
```
N = RL3( RL2( RL1(X) ) )  
t1 = RL1(X)  
t2 = RL2(t1)  
Y = RL3(t2)
```

```
N = ((X W1+B1 ) W2+B2 )W3+B3  
t1 = X W1+B1  
t2 = t1 W2 + B2  
Y = t2 W3 + B3
```

```
def forward(x,RL1,RL2,RL3)  
    t1=RL1.forward(x)  
    t2=RL2.forward(t1)  
    Y =RL3.forward(t2)  
    return Y
```

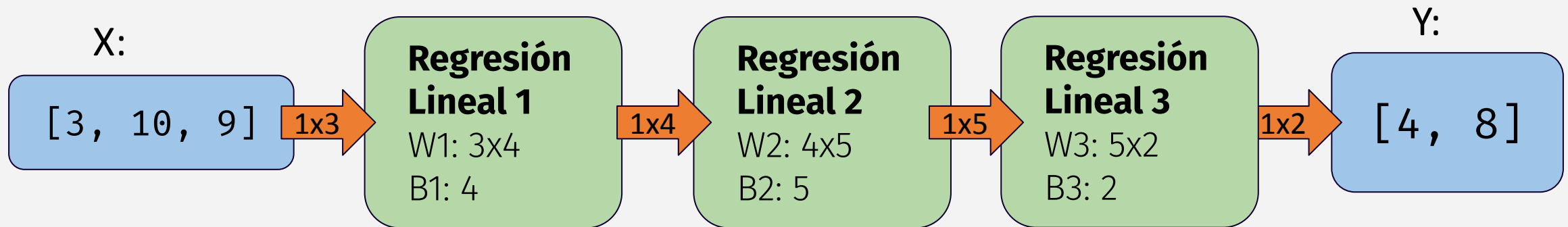
Redes Neuronales - Función de error

- Función de error
 - Considerarla como una **capa** más
 - Capa especial
 - No se usa para predecir
 - Solo para entrenar



Redes Neuronales - Funciones de activación

- Pero esto no va a funcionar..
 - 3 Regresiones = 1 Regresión
 - N Regresiones = 1 Regresión



W y V son matrices constantes
=> W.V es otra matriz.

W es una matriz y B es un
vector => W.B es otro vector

$$Y = ((X W1 + B1) W2 + B2) W3 + B3$$

$$Y = ((X W1 W2 + B1 W2) + B2) W3 + B3$$

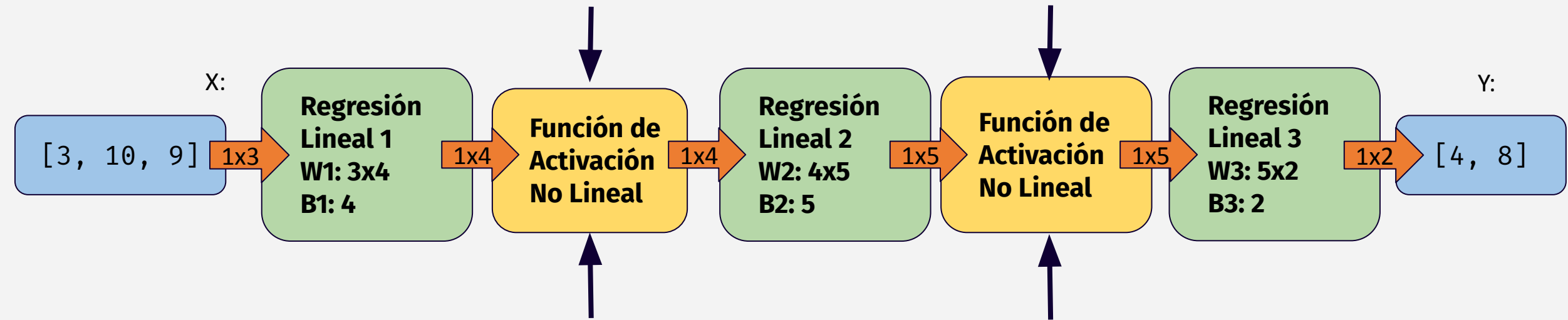
$$Y = (X W4 + B4) W3 + B3$$

$$Y = (X W4 W3 + B4 W3) + B3$$

$$Y = X W5 + B5$$

Redes Neuronales - Funciones de activación

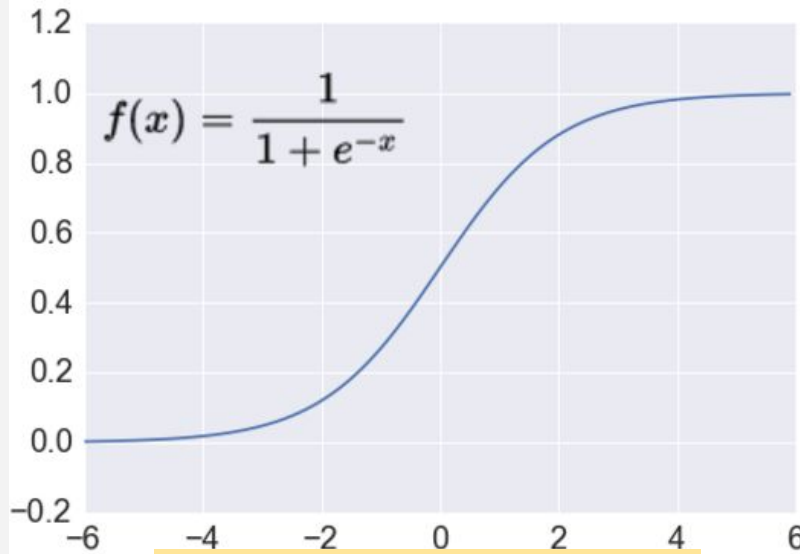
- Permiten estimar con funciones **no lineales**
- Cualquier función, siempre que sea derivable
- $\sin(x)$, $\cos(x)$, $\tan(x)$
- $f(x)=x^2$, $g(x)=x^{25*5-584}$, etc



- Funciones populares: $\text{ReLU}(x)$, $\text{Tanh}(x)$ o $\text{Sig}(x)$
- Capas SIN parámetros
 - No se entrenan!

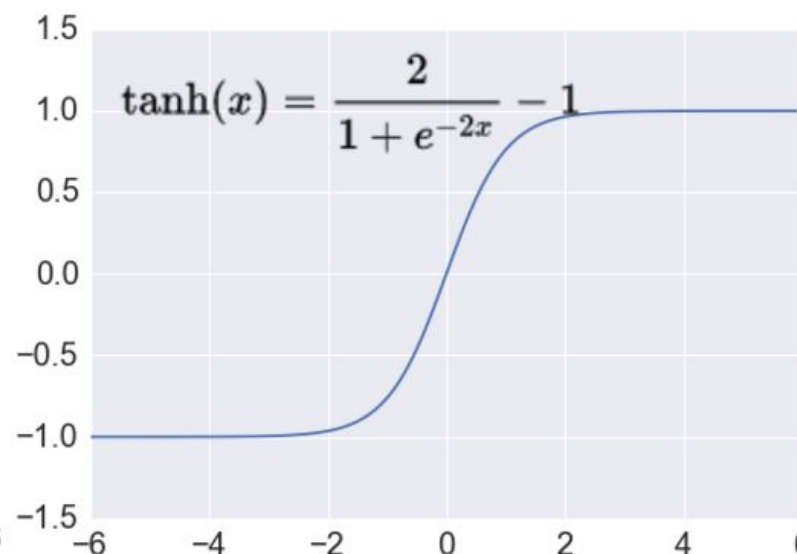
Redes Neuronales - Funciones de Activación

Sigmoid



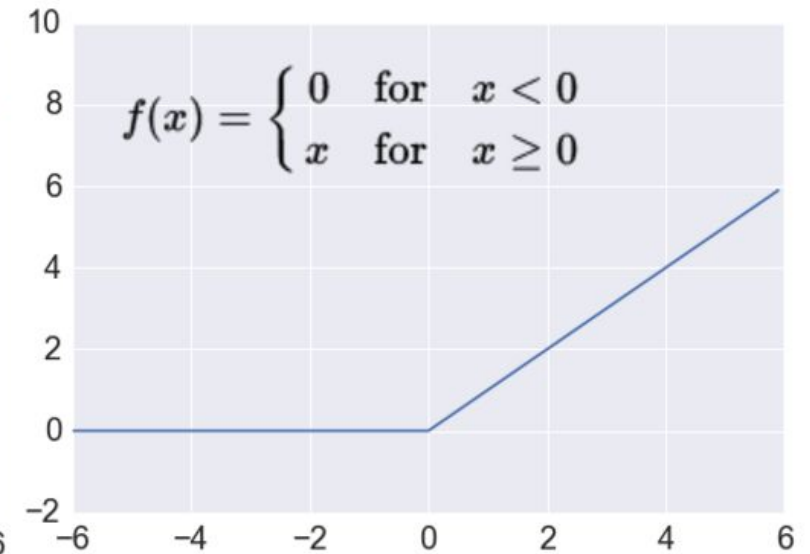
```
def sigmoid(x):  
    d=1+exp(-x)  
    return 1/d
```

TanH



```
def TanH(x)  
    s=sigmoid(x)  
    return (x*2)-1
```

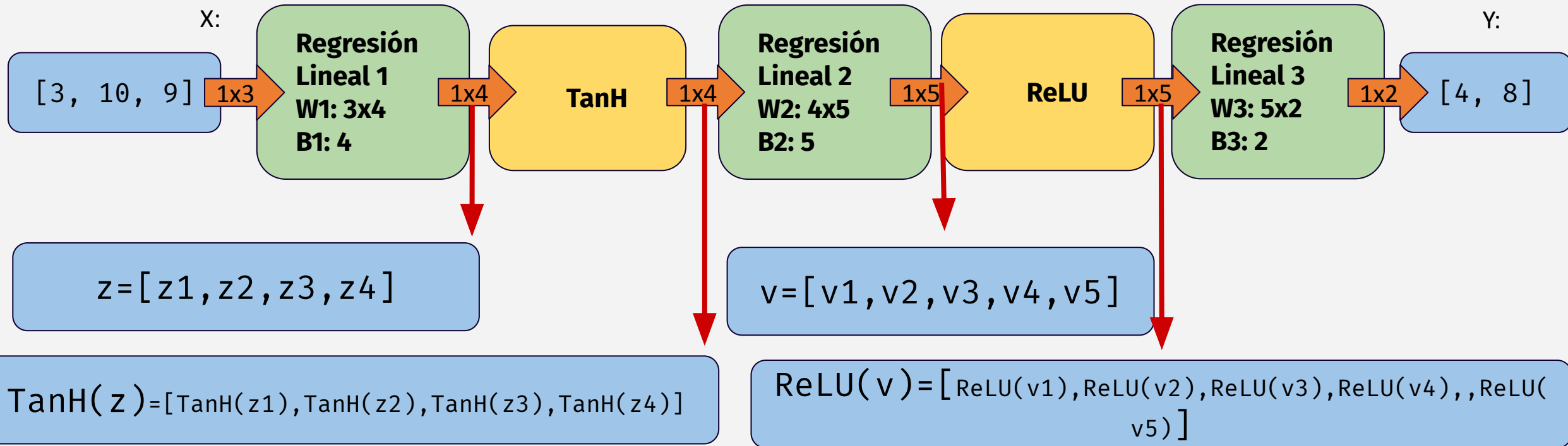
ReLU



```
def ReLU(x):  
    if x<=0:  
        return 0  
    else:  
        return x
```

- Sigmoid = la misma que regresión logística
- TanH = (Sigmoid*2)-1
- ReLU = 0 en los negativos, lineal en los positivos
- Generalmente se usa ReLU o variantes, más fácil de entrenar

Redes Neuronales - Funciones de activación



```
def TanH_vector(z):  
    res=np.zeros_like(z)  
    for i in range(len(z)):  
        res[i]=TanH(z[i])  
    return res
```

```
def ReLU_vector(v):  
    res=np.zeros_like(v)  
    for i in range(len(v)):  
        res[i]=ReLU(v[i])  
    return res
```

Regresión Con Redes + Keras

```
x,y=cargar_dataset()  
nx,d_in  = x.shape # x tiene tamaño n x d_in  
ny,d_out = y.shape # y tiene tamaño n x d_out  
import keras
```

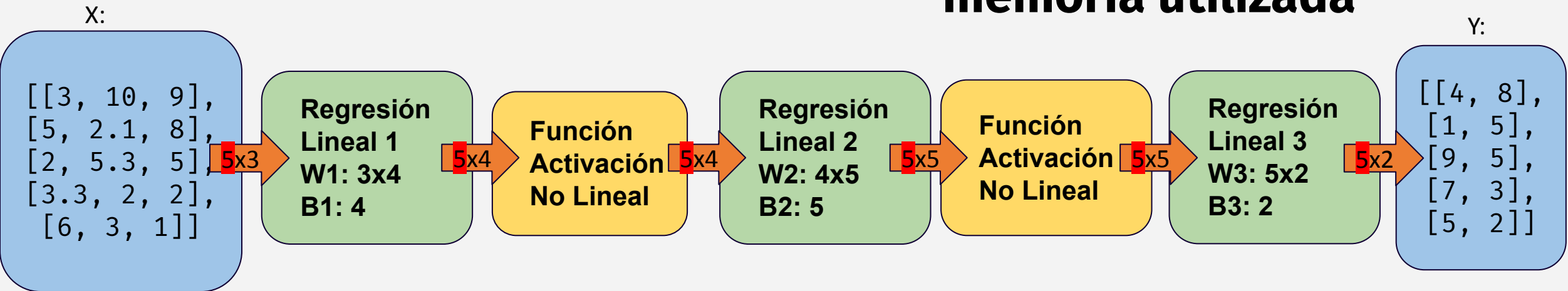
```
model=keras.Sequential()  
model.add(keras.Dense(4,input_shape=[d_in],activation='tanh'))  
model.add(keras.Dense(5,activation='relu'))  
model.add(keras.Dense(d_out,activation='none'))  
model.compile(loss='mse', # error cuadrático medio  
              optimizer='sgd') # descenso de gradiente  
history = model.fit(x,y,epochs=100,batch_size=32)  
y_predicted=model.predict(x)
```

- Los valores 4 y 5 son arbitrarios
- La cantidad de capas también
- Son hiperparámetros

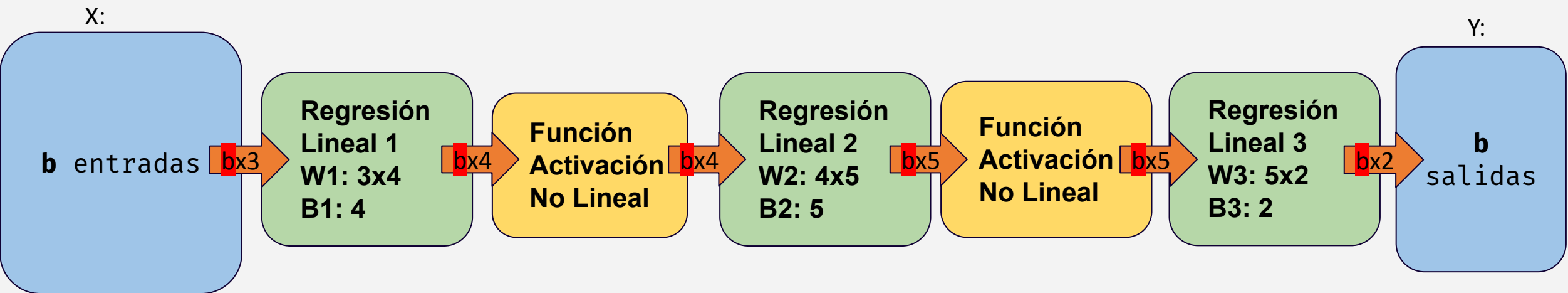
Redes Neuronales - Ejecución por lotes

Ejemplo con batch_size=5

**Importante para conocer
memoria utilizada**

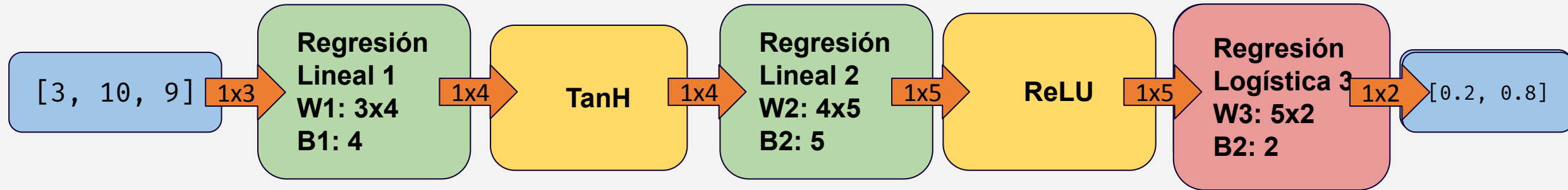


En general, $b \times n$ (b = batch_size)



Redes Neuronales - Clasificación

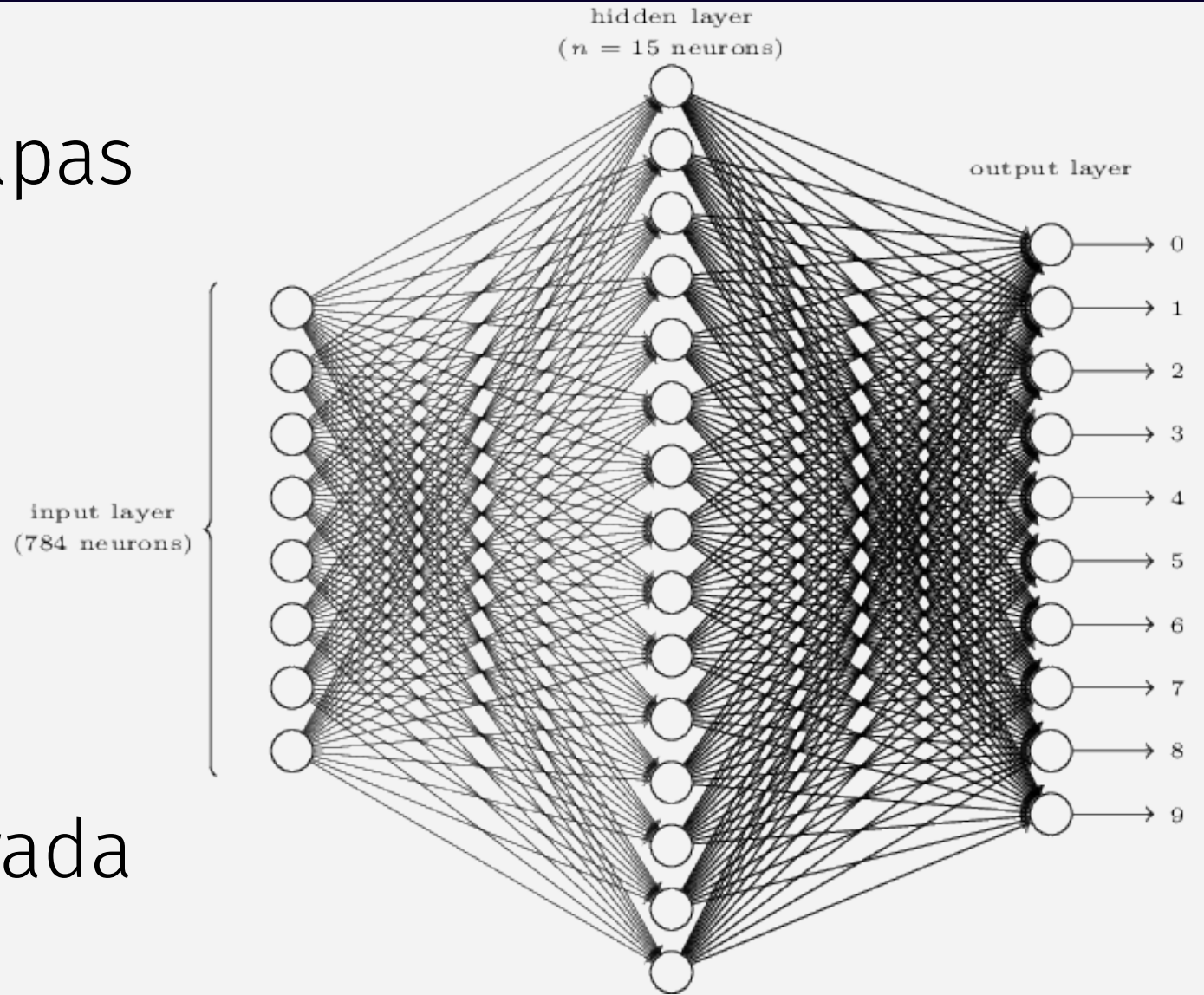
Modificar la red anterior para clasificación con 2 clases:



- Capas intercambiables
 - Pero verificar tamaños
- Cambio última capa:
 - Cambia el tipo de dato de salida
- Última capa se llama cabeza o head.

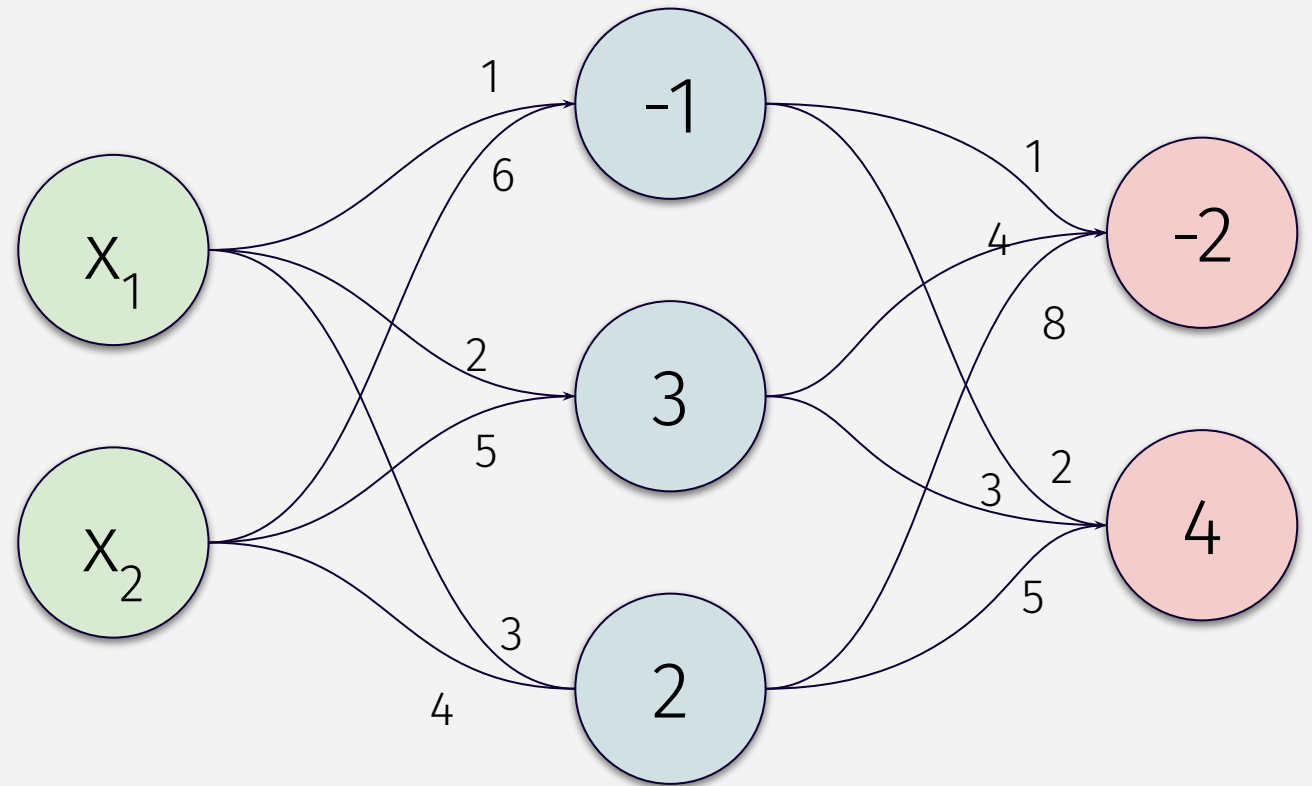
Capa Densa o Fully Connected o Reg. Lineal

- Nomenclatura
 - Viene de red de 2 capas
- Valores = Neuronas
 - De entrada
 - Ocultas
 - Salida
- $y = w x + b$
 - x : N entradas
 - y : M salidas
 - w conecta cada entrada con cada salida
 - tamaño $N \times M$



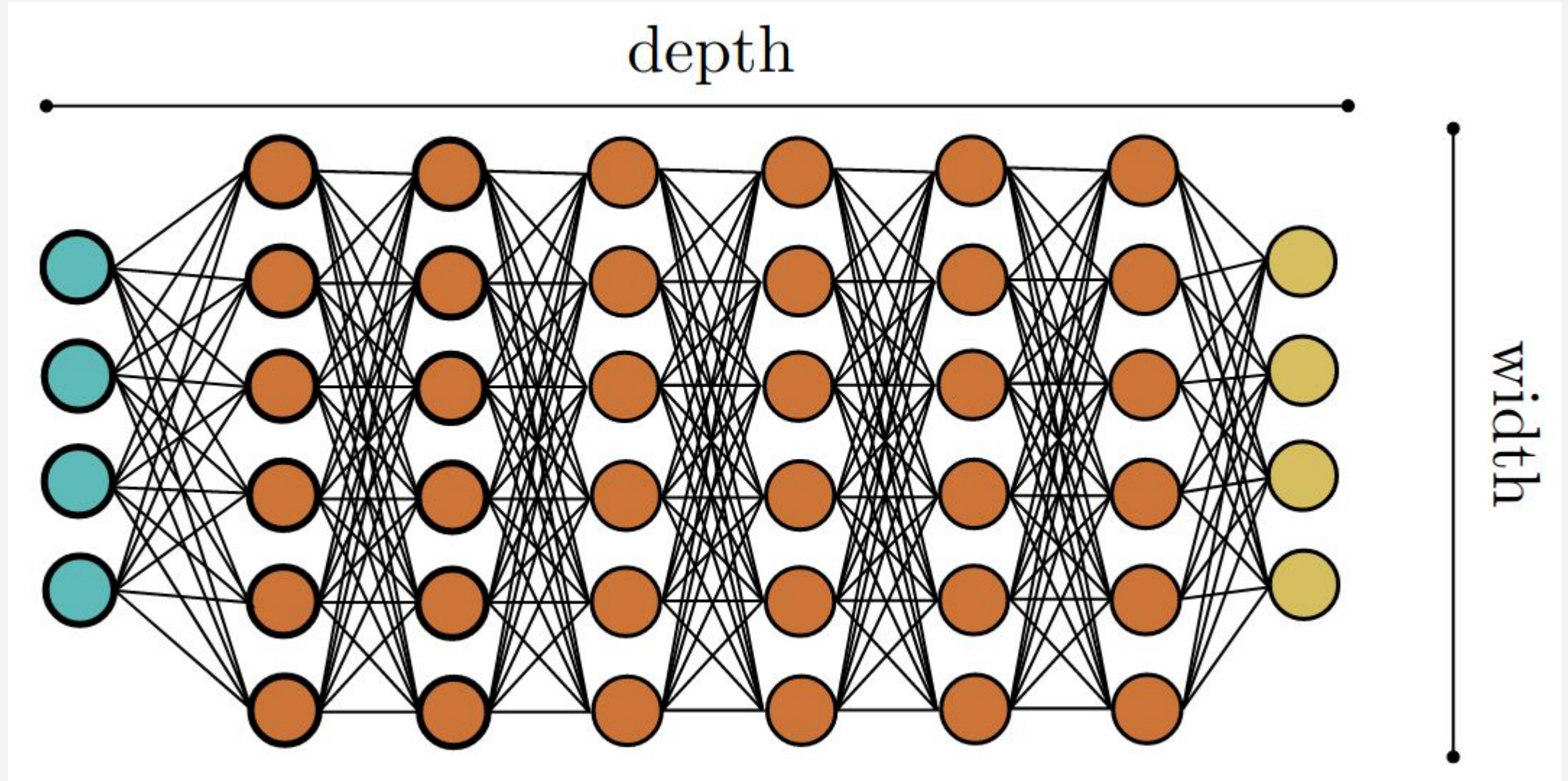
Capa Densa o Fully Connected o Reg. Lineal

- Ejemplo red 2 capas
 - 2 entradas
 - 3 ocultas
 - 2 salidas
- $f(x) = (xw_1 + b_1)w_2 + b_2$
 - $w_1 = \begin{bmatrix} 1, 2, 3 \\ 6, 5, 4 \end{bmatrix}$
 - $b_1 = [-1, 3, 2]$
 - $w_2 = \begin{bmatrix} 1, 2 \\ 4, 3 \\ 8, 5 \end{bmatrix}$
 - $b_2 = [-2, 4]$



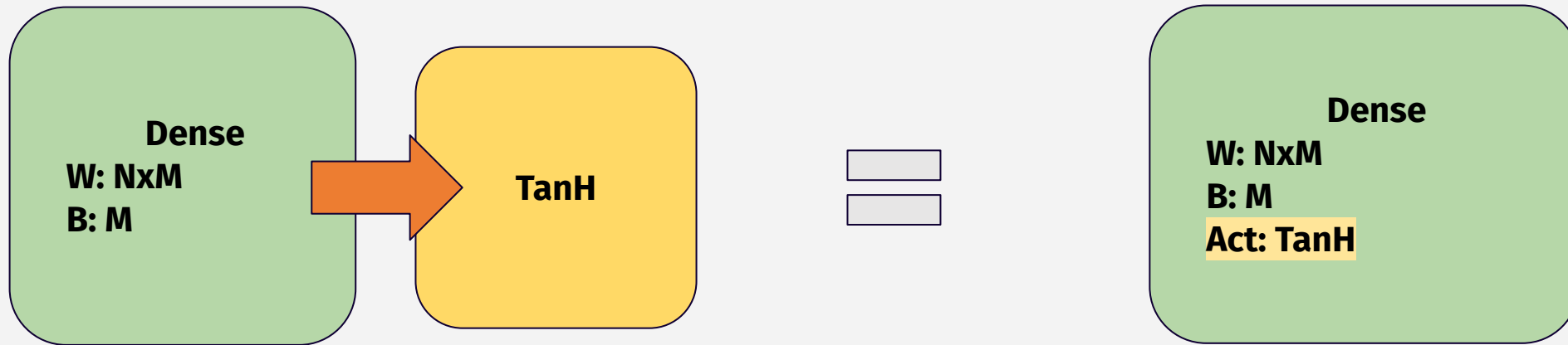
- Aristas
 - Valores de w_1 y w_2
- Nodos
 - Valores de b_1 y b_2

Profundidad vs Ancho



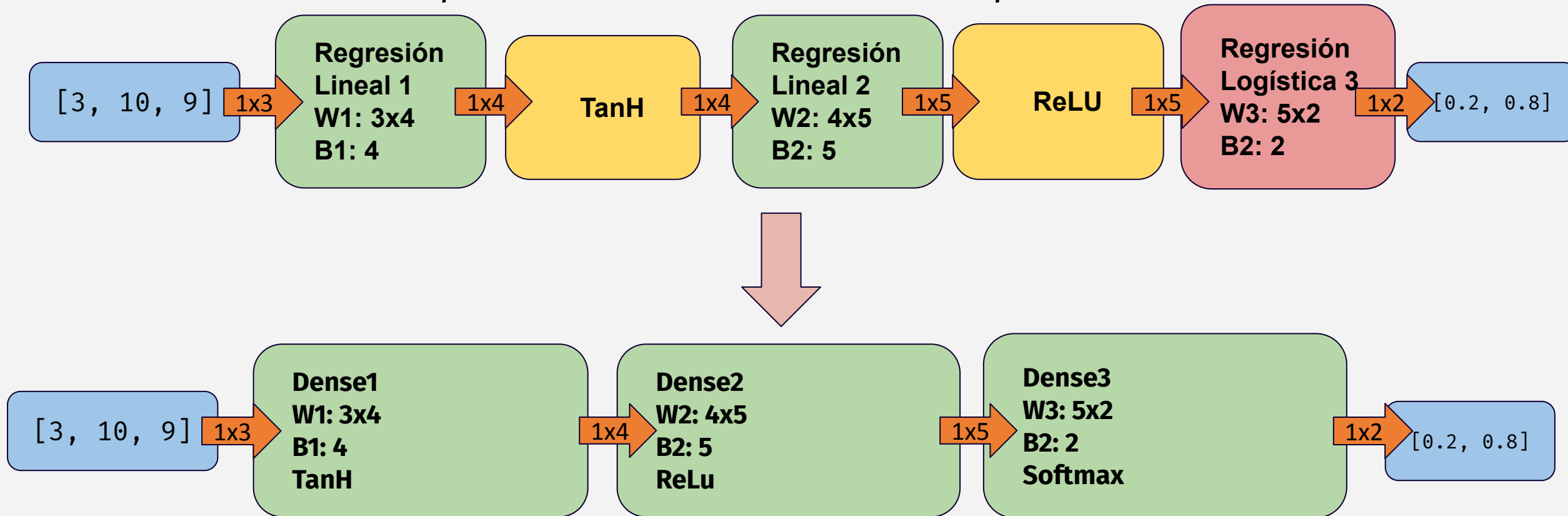
Densas con activación vs Densas y activación

- Activación “dentro” de la capa
 - Equivalente
 - Cómodo



Redes Neuronales - Términos

- Regresión Lineal
- Regresión Logística
 - Reservados para modelos de 1 capa



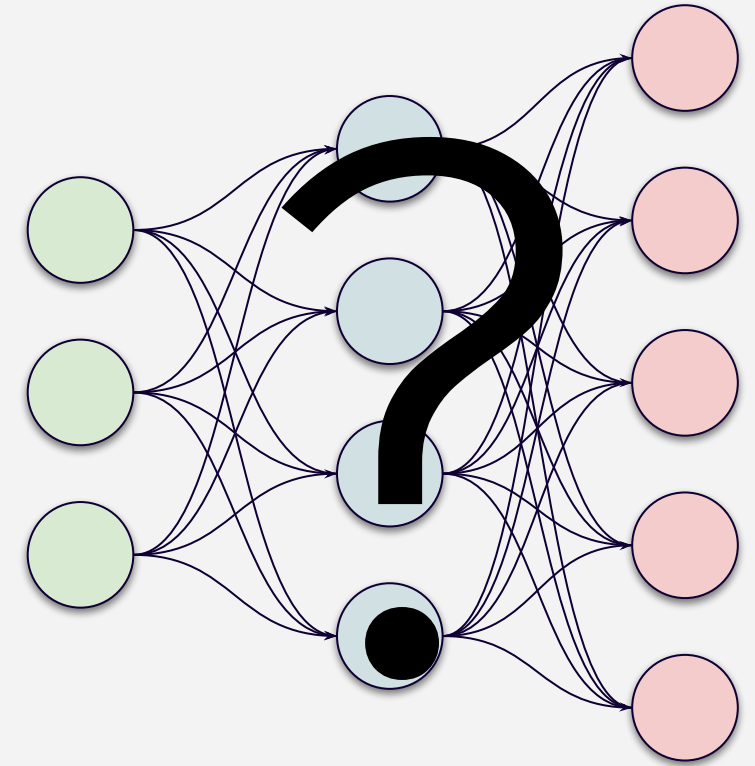
Clasificación Con Redes + Keras

```
x,y=cargar_dataset(one_hot=True)
nx,d_in  = x.shape # x tiene tamaño n x d_in
ny,d_out = y.shape # y tiene tamaño n x d_out
import keras

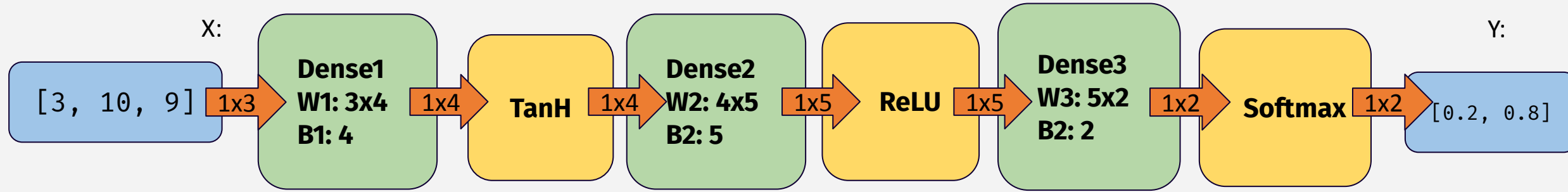
model=keras.Sequential()
model.add(keras.Dense(4,input_shape=[d_in],activation='tanh'))
model.add(keras.Dense(5,activation='relu'))
model.add(keras.Dense(d_out,activation='softmax'))
model.compile(loss='categorical_crossentropy', # ent cruz
              optimizer='sgd', # descenso de gradiente
              metrics='accuracy')
history = model.fit(x,y,epochs=100,batch_size=32)
y_predicted=model.predict(x)
```

Diseño de redes

- Dado un problema X
 - ¿Cuántas capas?
 - ¿Cuántas neuronas por capa?
 - ¿Qué funciones de activación?
- No hay **un** óptimo
 - Varias combinaciones funcionan
- ¿Cómo diseñar?
 - Experiencia previa y literatura
 - Búsqueda de hiperparámetros
 - #capas, #neuronas, tipos de capa



Redes Neuronales - Resumen

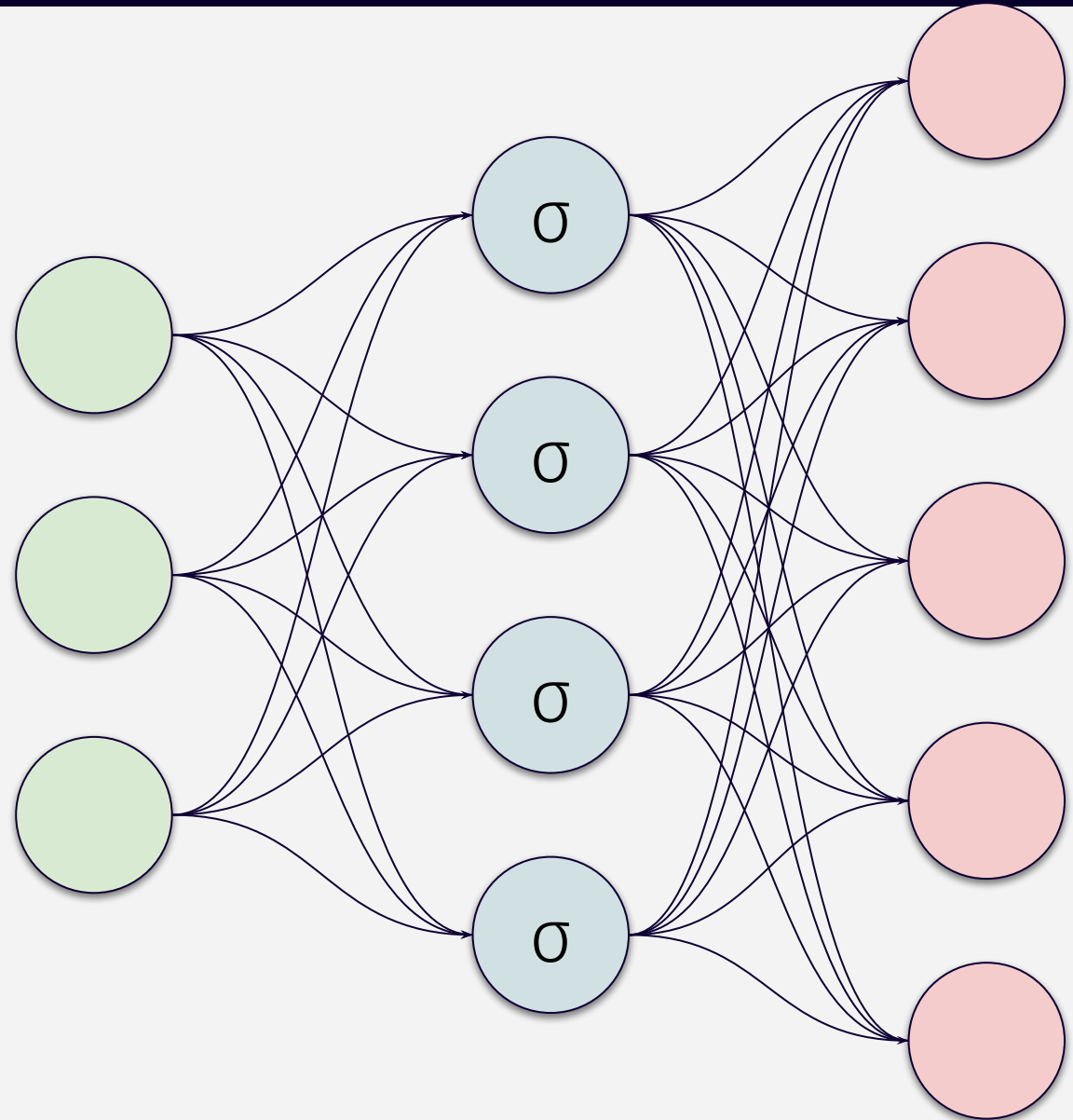


- Transforman vectores de entrada en vectores de salida
- Regresión Lineal/Logística + Funciones de activación
 - Transformaciones no lineales
 - Mayor poder de clasificación/regresión
- Capas modulares
 - Combinar de cualquier forma => *arquitecturas* o *topologías*
- Descenso de gradiente para optimizar

No linealidad en Redes
Neuronales.
Aproximación Universal

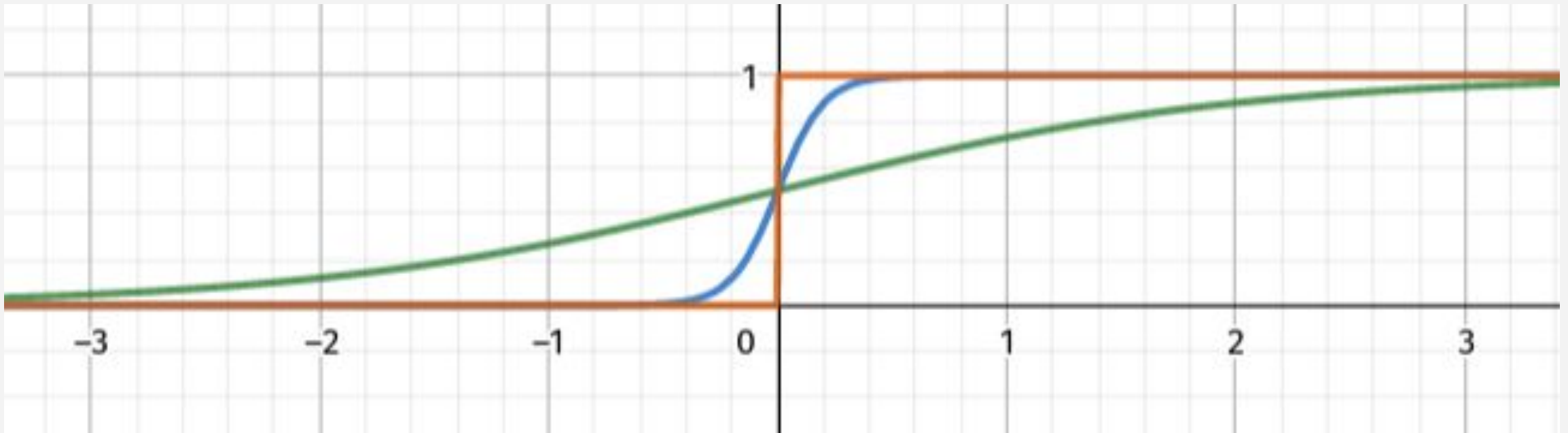
Teorema(s) de Cybenko

- Datos
 - Red
 - 2 capas
 - Fully connected
 - Act. sigmoidea (σ)
 - Conj. de datos
 - Función de error
- Aumentando #parámetros
 - Decrementa error
 - de forma arbitraria
- Teoremas
 - Para otras f. activación



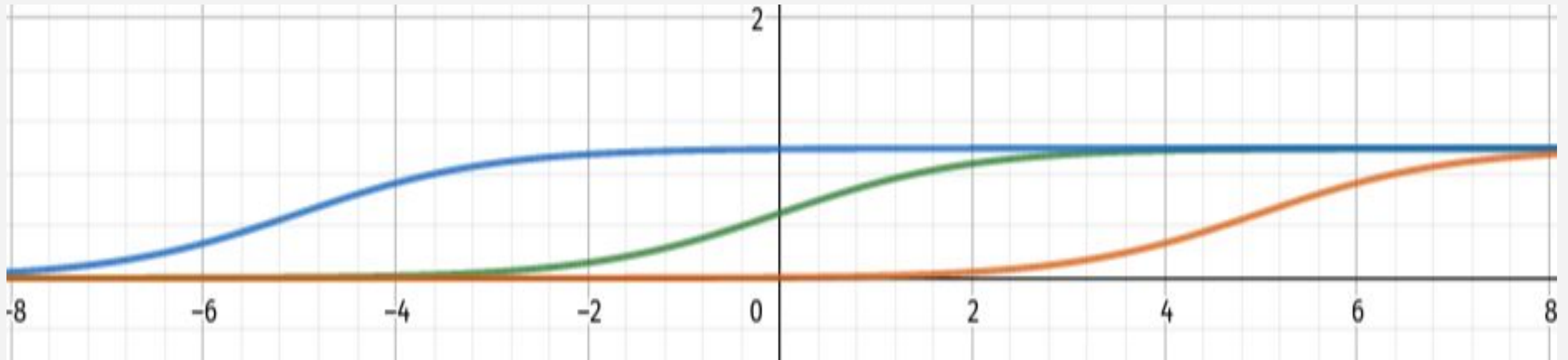
Función sigmoidea

- Sigmoidea
 - $\sigma(x) = 1 / (1+e^{-x})$
 - $\sigma(x) = 1 / (1+e^{-(wx+b)})$
- Variando w y b
 - Cambia salida
- w varia pendiente
 - $\sigma(x)$
 - $\sigma(10x)$
 - $\sigma(1000x)$ (función *step*)



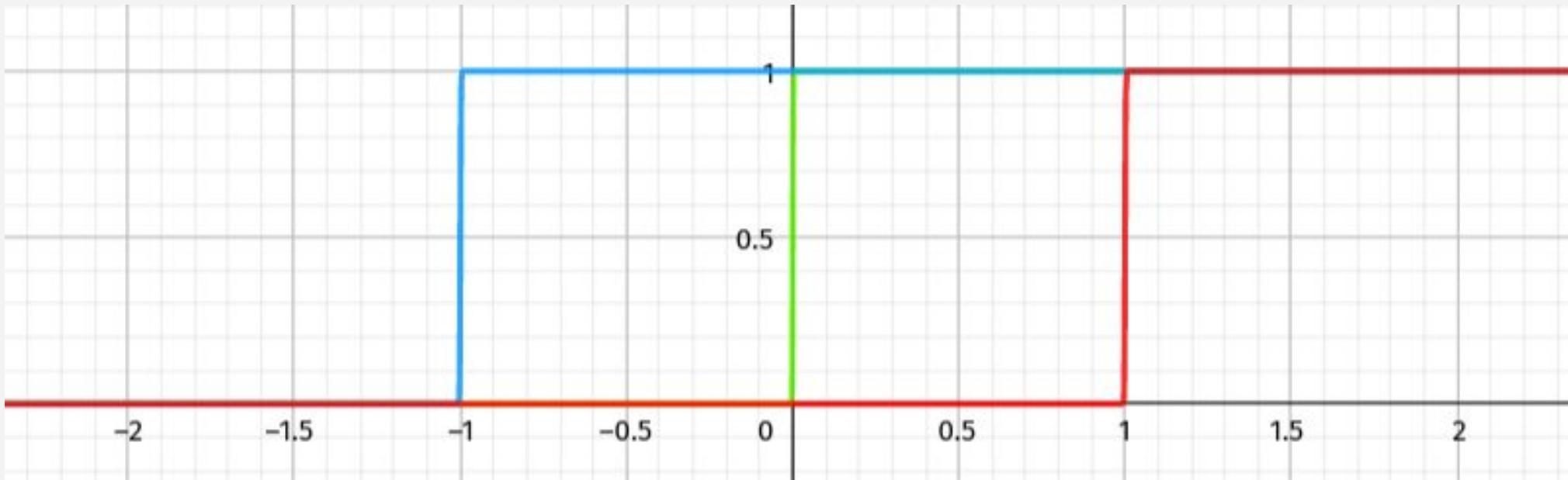
Función sigmoidea

- Sigmoidea
 - $\sigma(x) = 1 / (1+e^{-x})$
 - $\sigma(x) = 1 / (1+e^{-(wx+b)})$
- Variando w y b
 - Cambia salida
- b varia posición
 - $\sigma(x)$
 - $\sigma(x+10)$
 - $\sigma(x-10)$



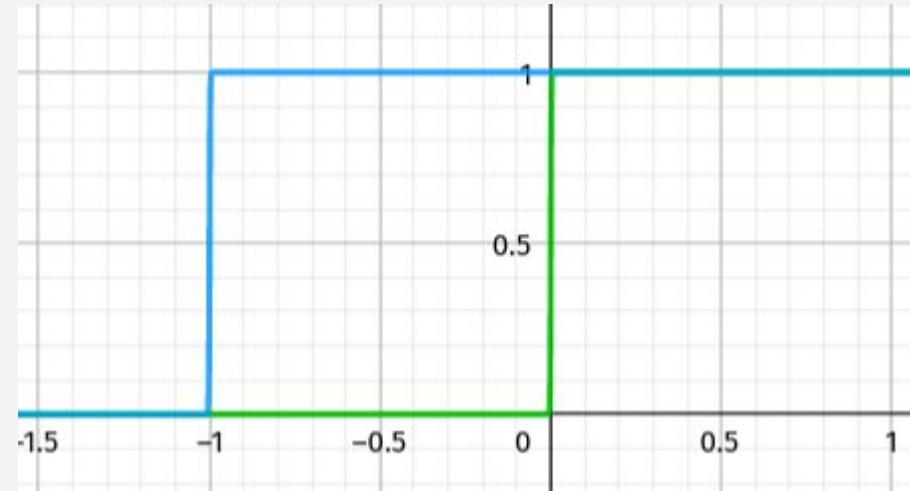
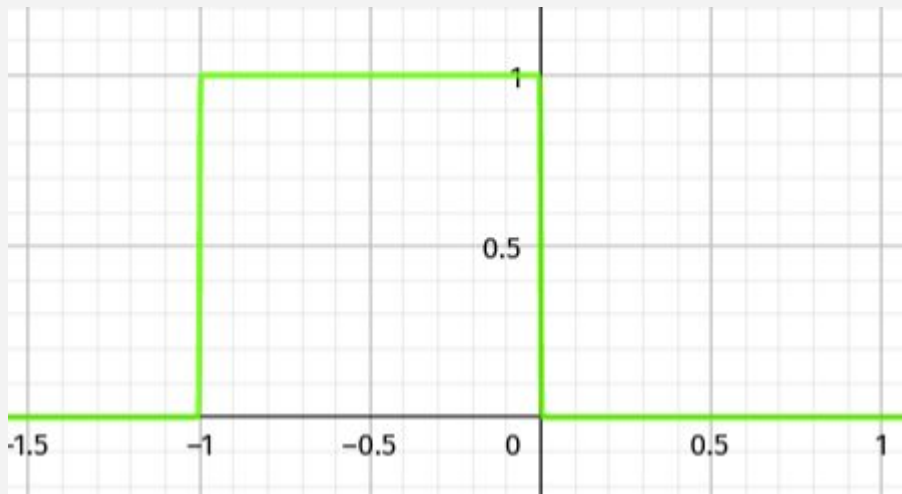
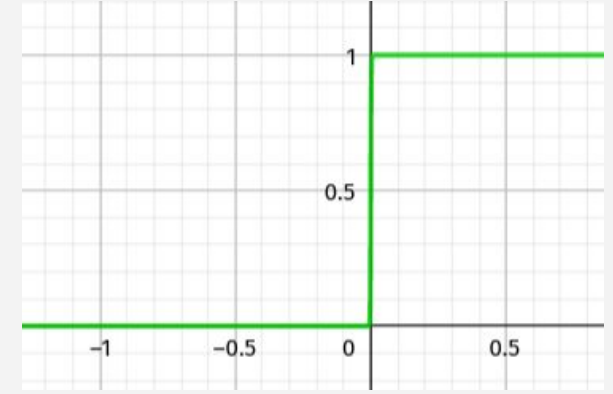
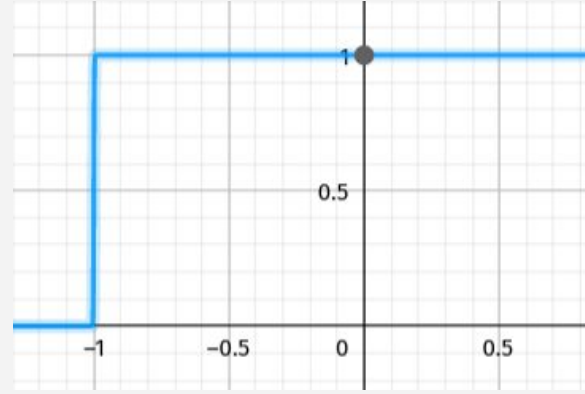
Función sigmoidea

- Sigmoidea
 - $\sigma(x) = 1 / (1+e^{-x})$
 - $\sigma(x) = 1 / (1+e^{-(wx+b)})$
- Variando w y b
 - Cambia salida
- w y b combinadas
 - $\sigma(1000x)$
 - $\sigma(1000x+1000)$
 - $\sigma(1000x-1000)$



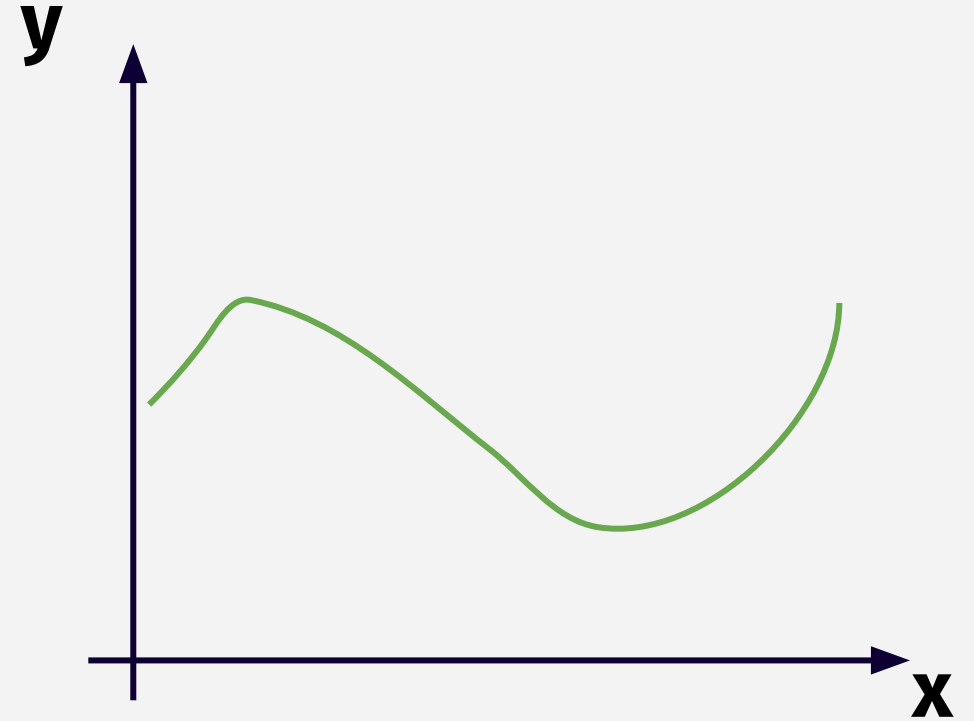
Truco: combinación de sigmoideas

- Sigmoidea
 - $\sigma(x) = 1 / (1 + e^{-x})$
- Selector de región
 - $\sigma(1000x)$
 - $\sigma(1000x+1000)$
 - $\sigma(1000x+1000) - \sigma(1000x)$



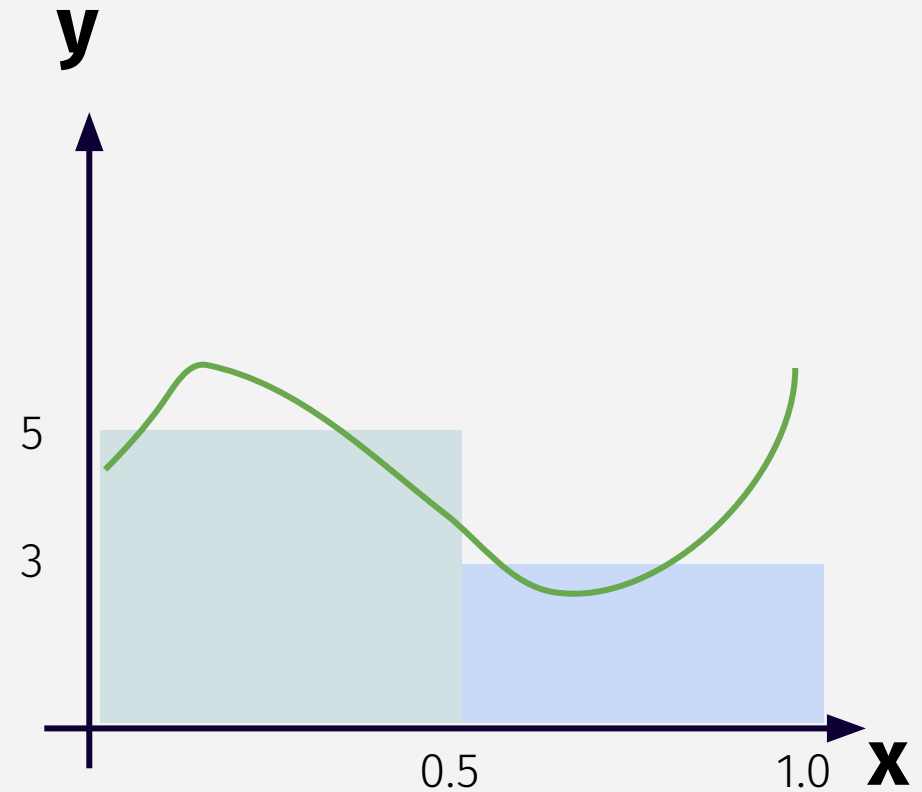
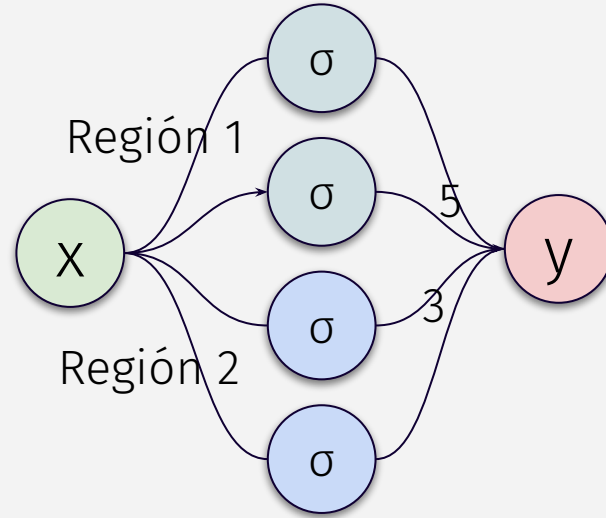
Teorema(s) de Cybenko - Intuición

- Problema de regresión 1D
 - Predecir **y** en base a **x**
- Red:
 - $y = w_2 \sigma(w_1 x + b_1) + b$
 - Equivalente
 - $o = \sigma(w_1 x + b_1)$
 - $y = w_2 o + b_2$
- Estrategia
 - $\sigma(w_1 x + b)$: selector de *región* de x
 - $w_2 o + b_2$: valor y para *región*



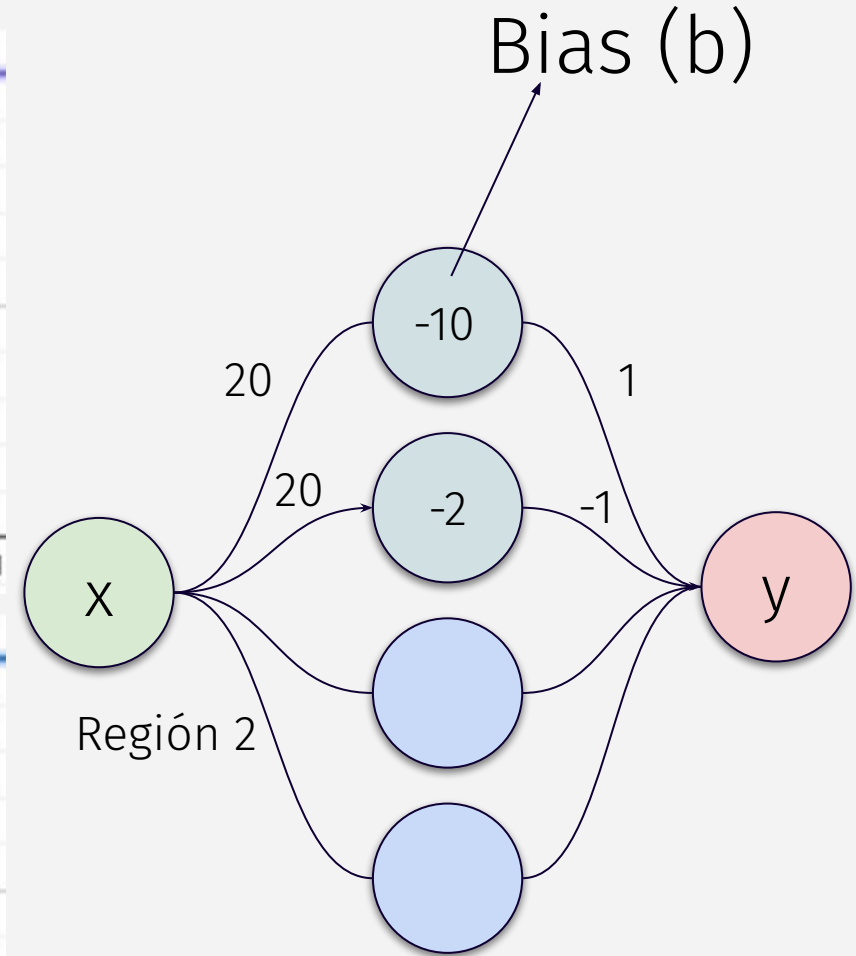
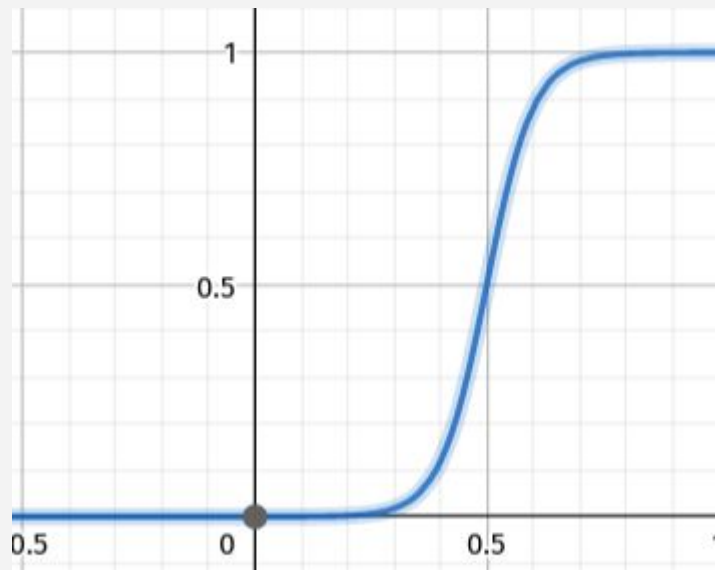
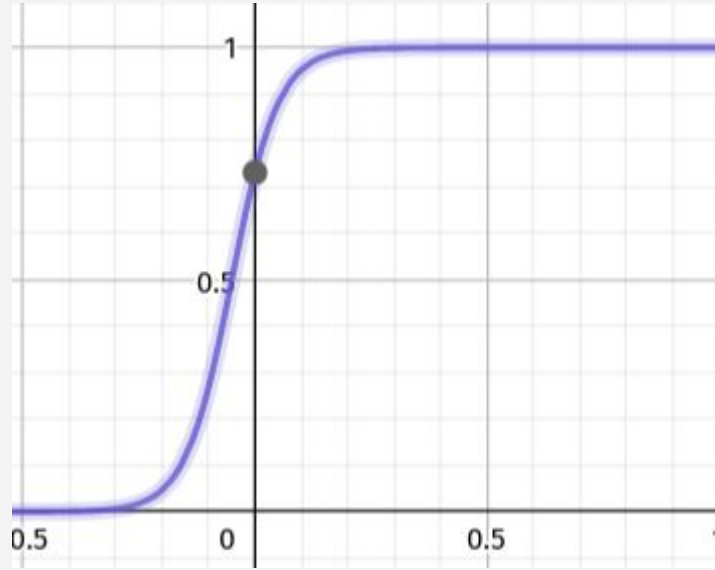
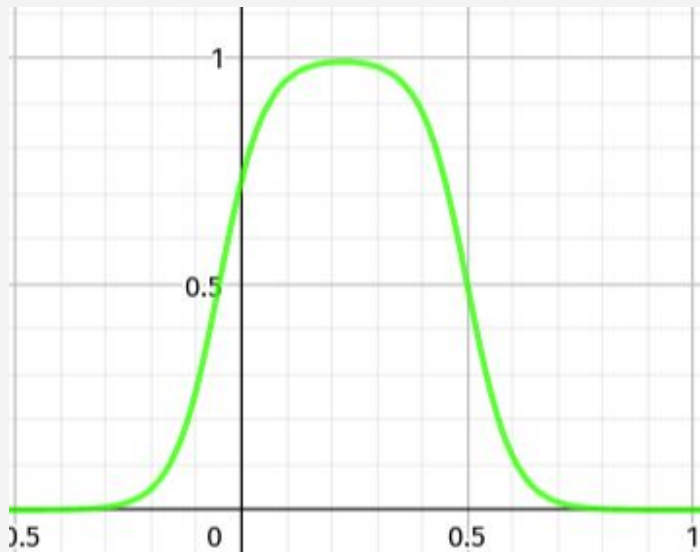
Teorema(s) de Cybenko - Intuición

- Ejemplo con 4 neuronas ocultas
 - w_1 es de 1×4
 - b_1 es de 4
 - w_2 es de 4×1
 - b_2 es de 1
- $4/2=2$ regiones
 - 2 neuronas por región
 - **Selectores** de región
- Región 1: $\mathbf{x}$ de 0 a 0.5
 - valor = 5
- Región 2: $\mathbf{x}$ de 0.5 a 1
 - valor = 3



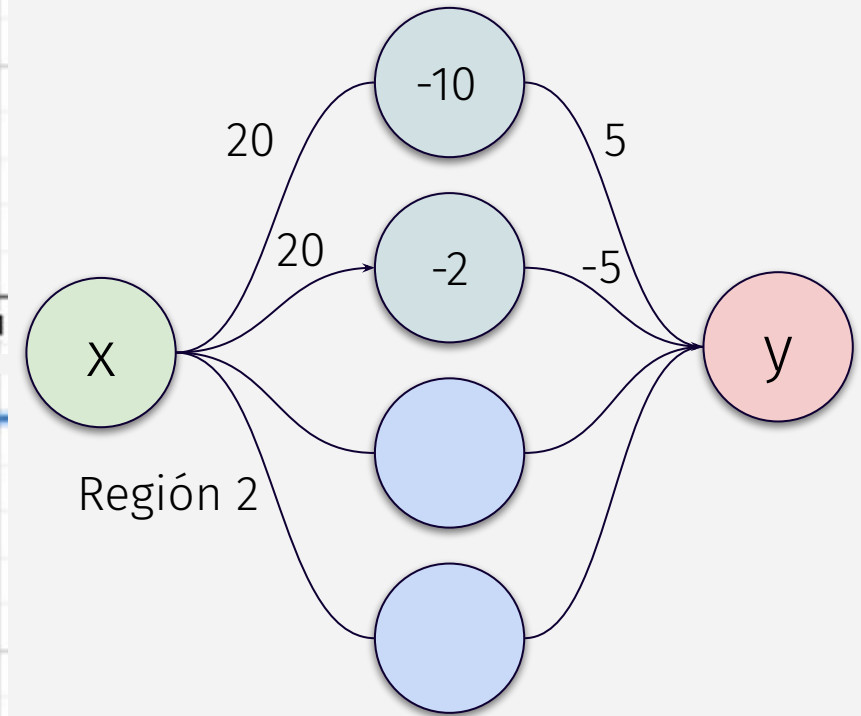
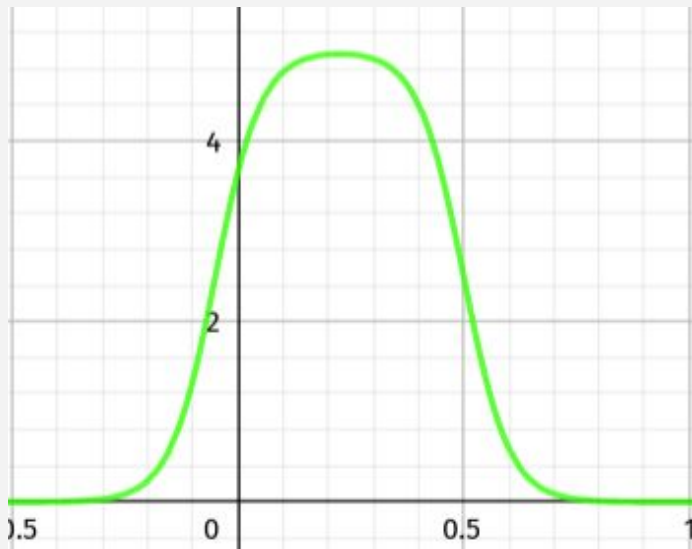
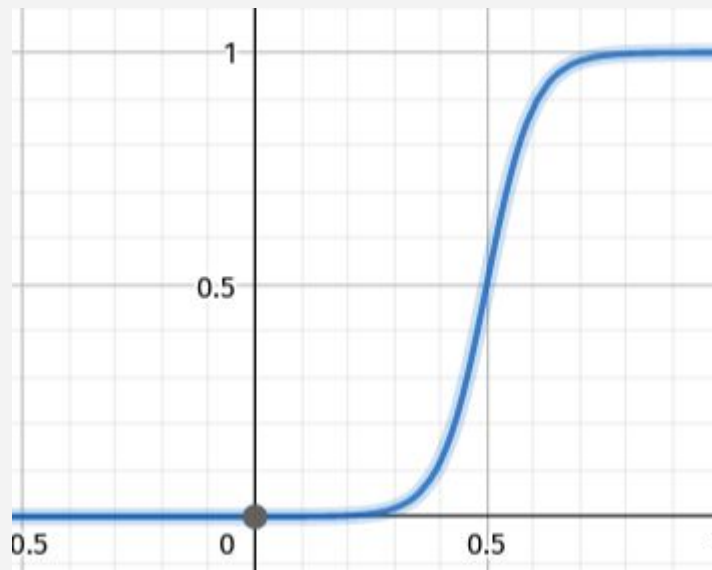
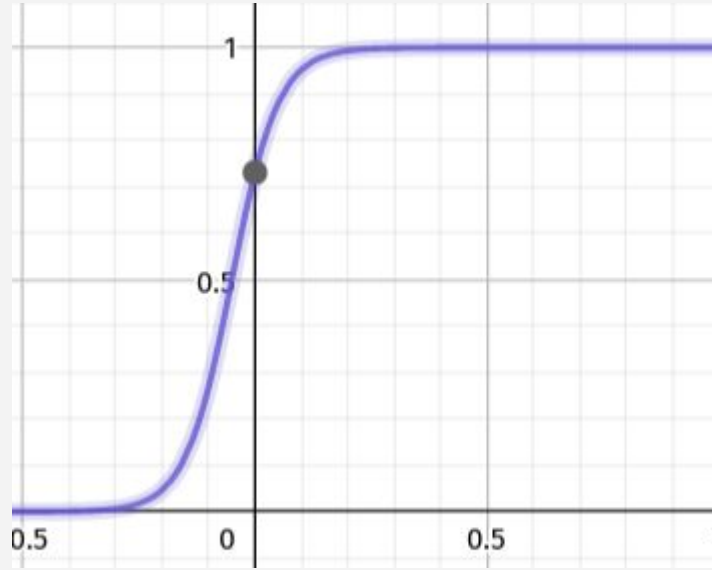
Selector de región 1

- Sigmoidea
 - $\sigma(x) = 1 / (1+e^{-x})$
 - $\sigma(x) = 1 / (1+e^{-(wx+b)})$
- $g(x) = \sigma(20x-10)$
- $h(x) = \sigma(20x-2)$
- $g(x)-h(x)$



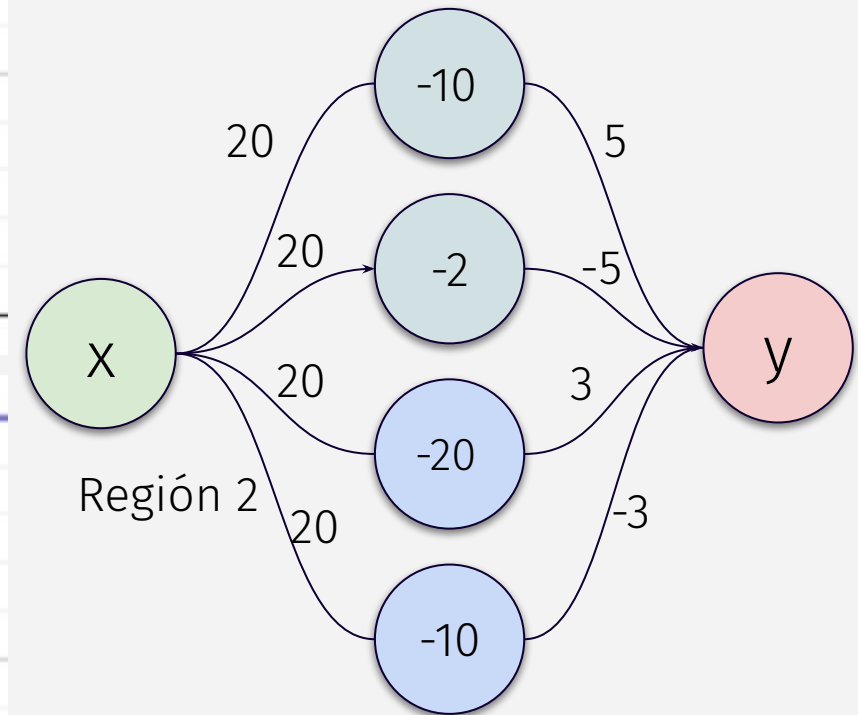
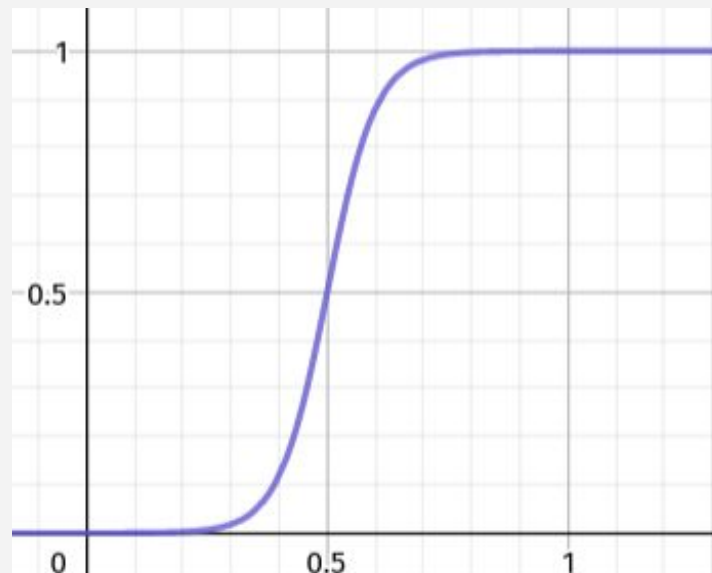
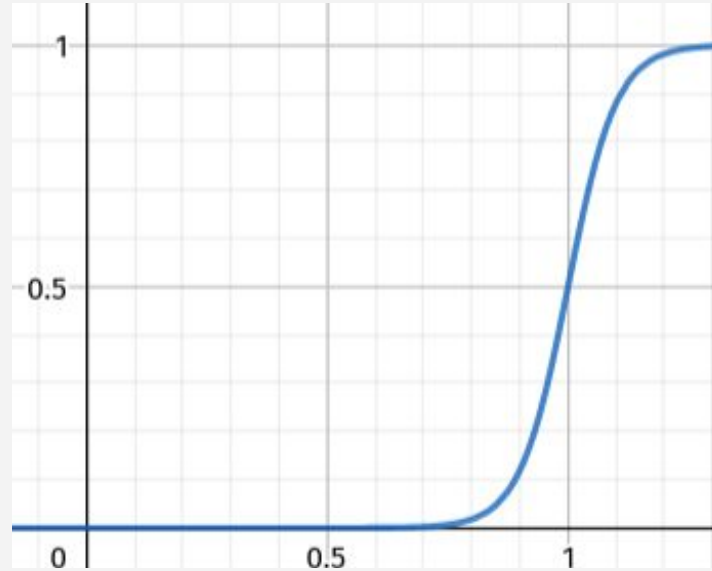
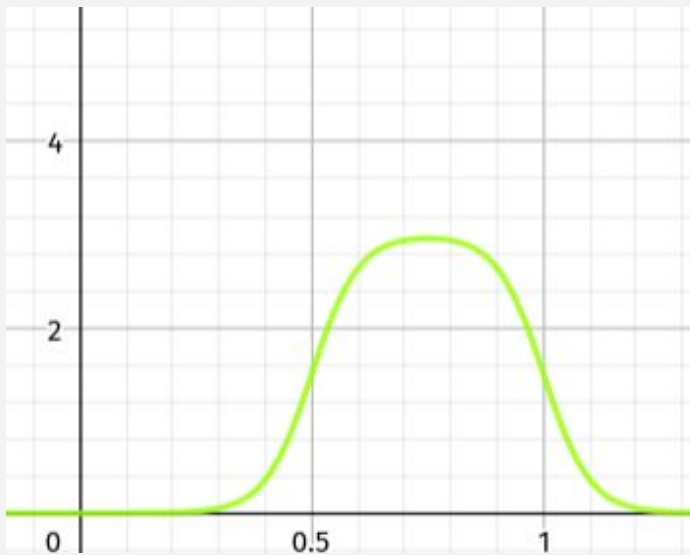
Valor para región 1: 5

- Sigmoidea
 - $\sigma(x) = 1 / (1+e^{-x})$
 - $\sigma(x) = 1 / (1+e^{-(wx+b)})$
- $g(x) = \sigma(20x-10)$
- $h(x) = \sigma(20x-2)$
- $5g(x) - 5h(x)$



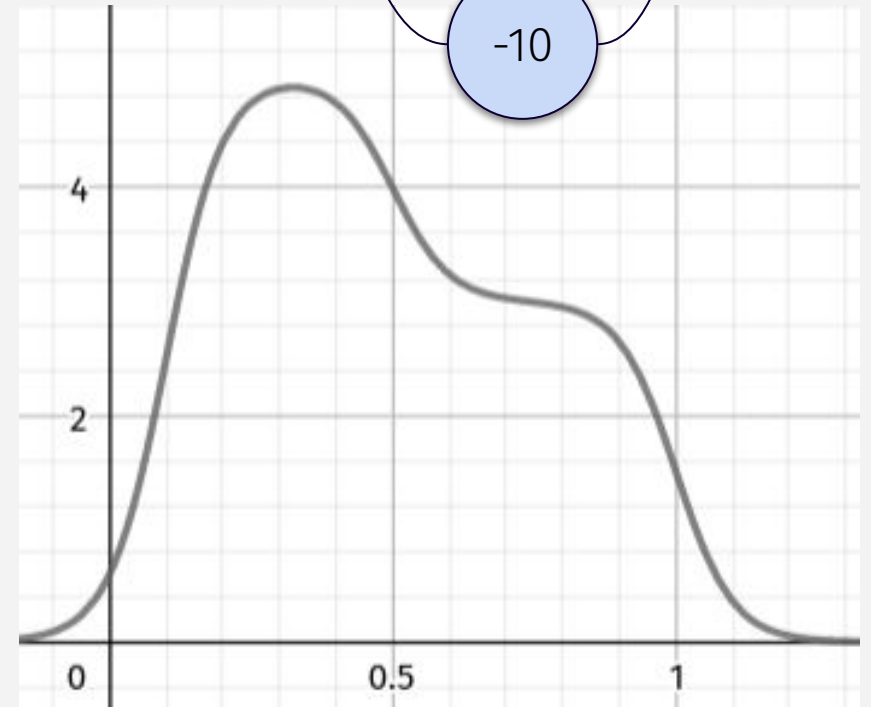
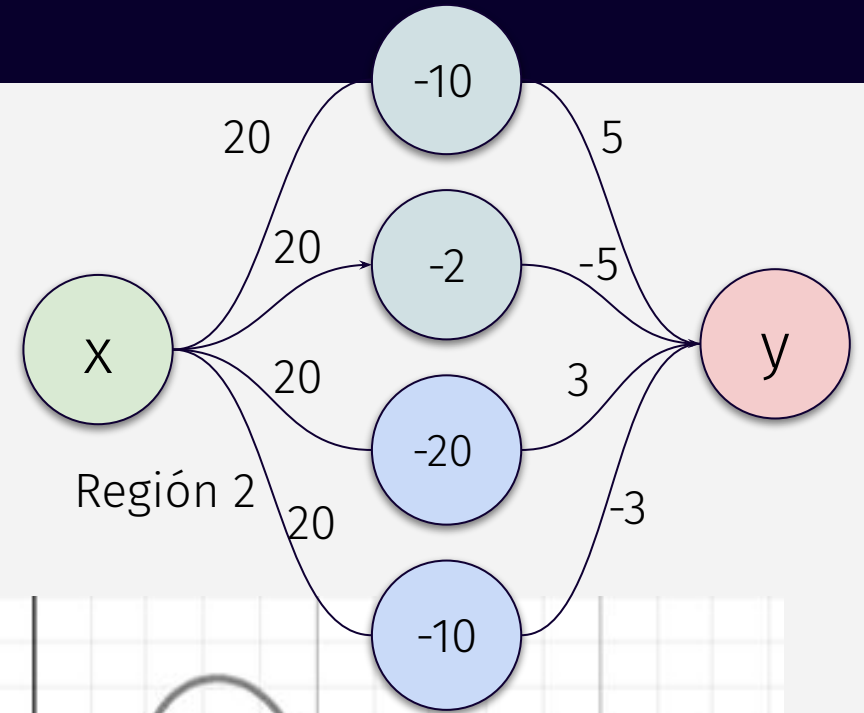
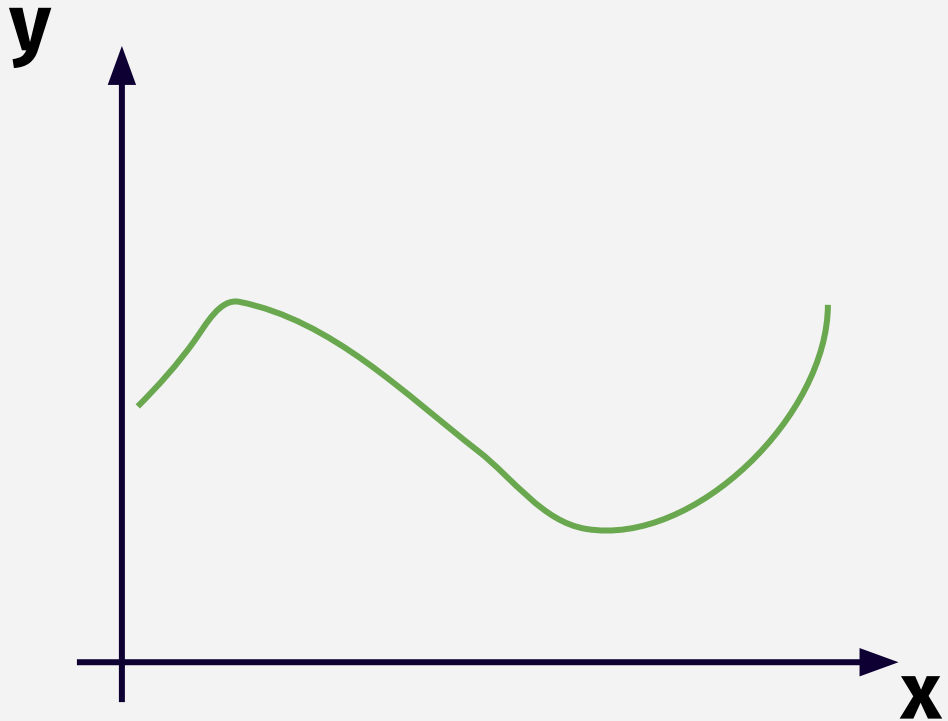
Selector región 2 (0.5 a 1) y valor (3)

- Sigmoidea
 - $\sigma(x) = 1 / (1+e^{-x})$
 - $\sigma(x) = 1 / (1+e^{-(wx+b)})$
- $g(x) = \sigma(20x-20)$
- $h(x) = \sigma(20x-10)$
- $3g(x) - 3h(x)$



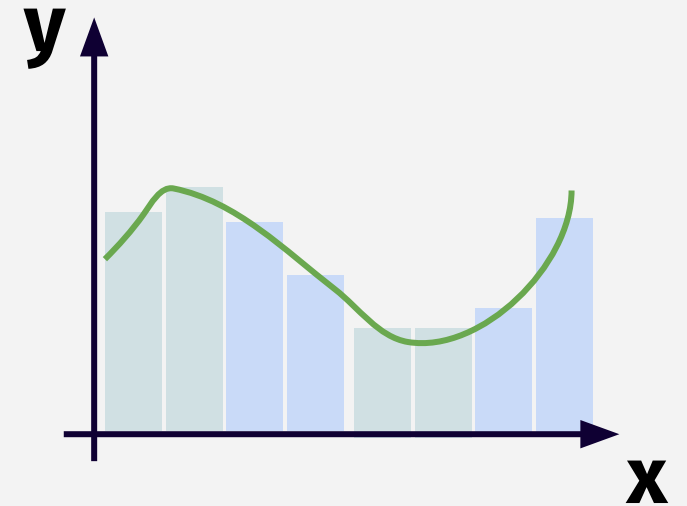
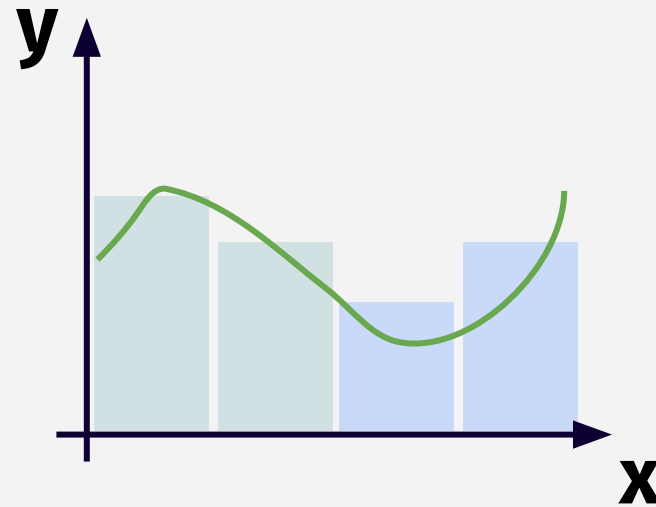
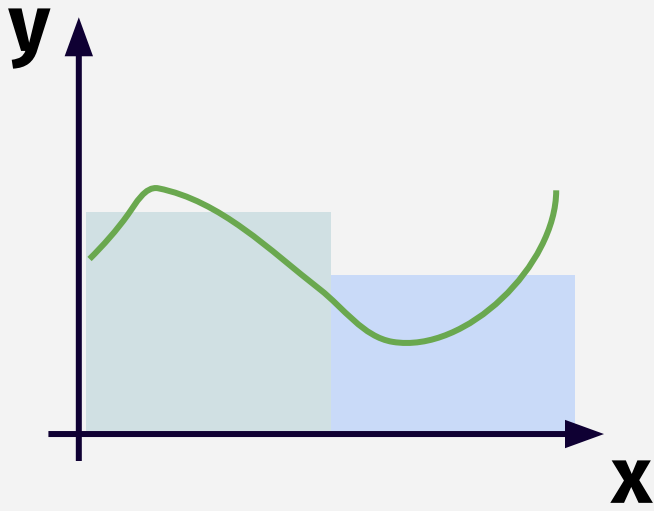
Función final

- $y = 5 \sigma(20x-10) - 5 \sigma(20x-2) + 3 \sigma(20x-20) - 3 \sigma(20x-10)$



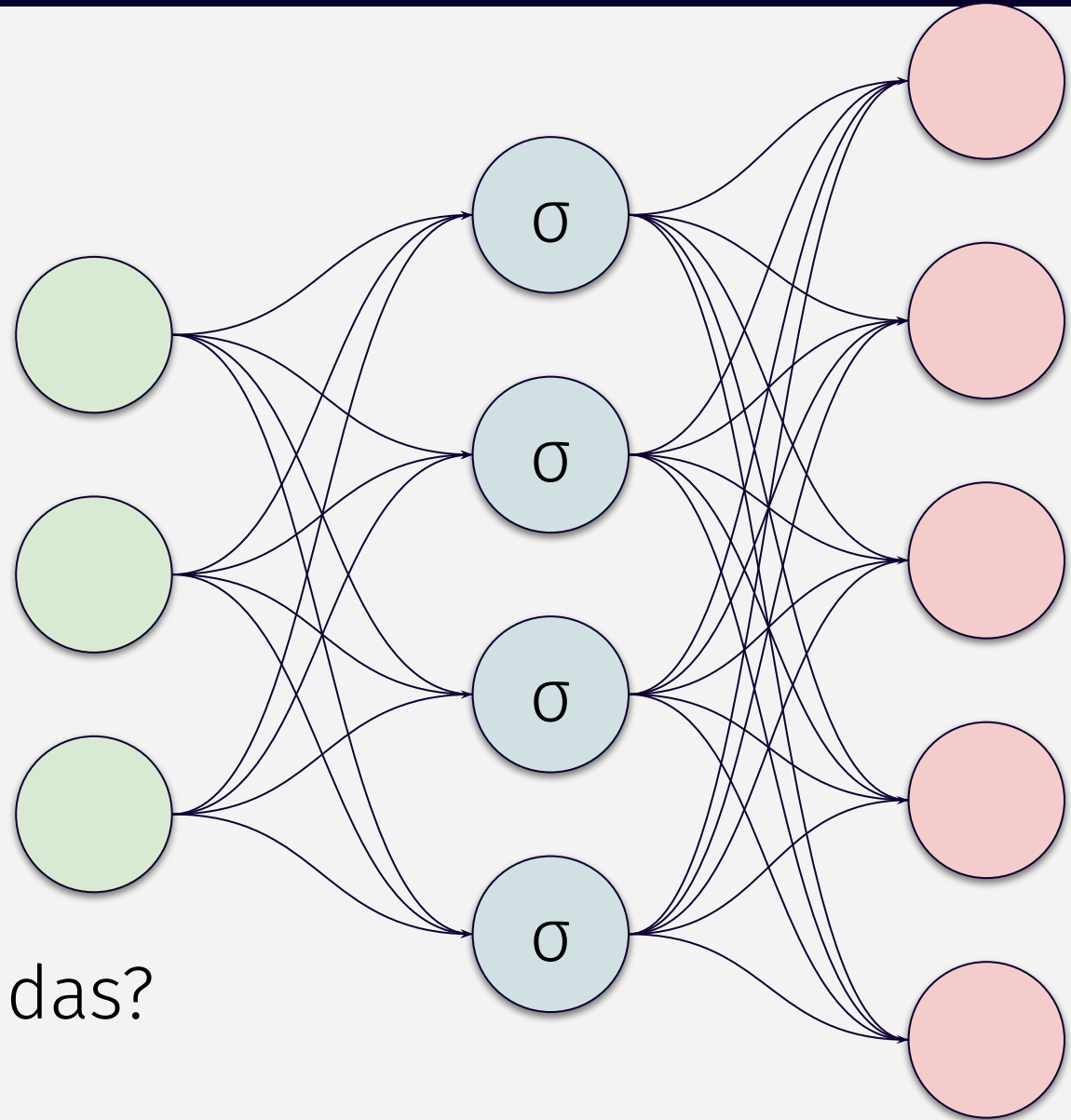
¿Usando más regiones?

- Más regiones
 - Más parámetros
- Aproximación arbitraria



Teorema(s) de Cybenko - Resumen

- Red 2 capas
 - Aproximador universal
- Capa 1
 - **Selectores** de región
- Capa 2
 - Asignan valor por región
- + regiones
 - + neuronas
 - + parámetros
 - + precisión
- ¿Otros problemas? ¿más salidas?
- ¿Reutilización de cómputo?

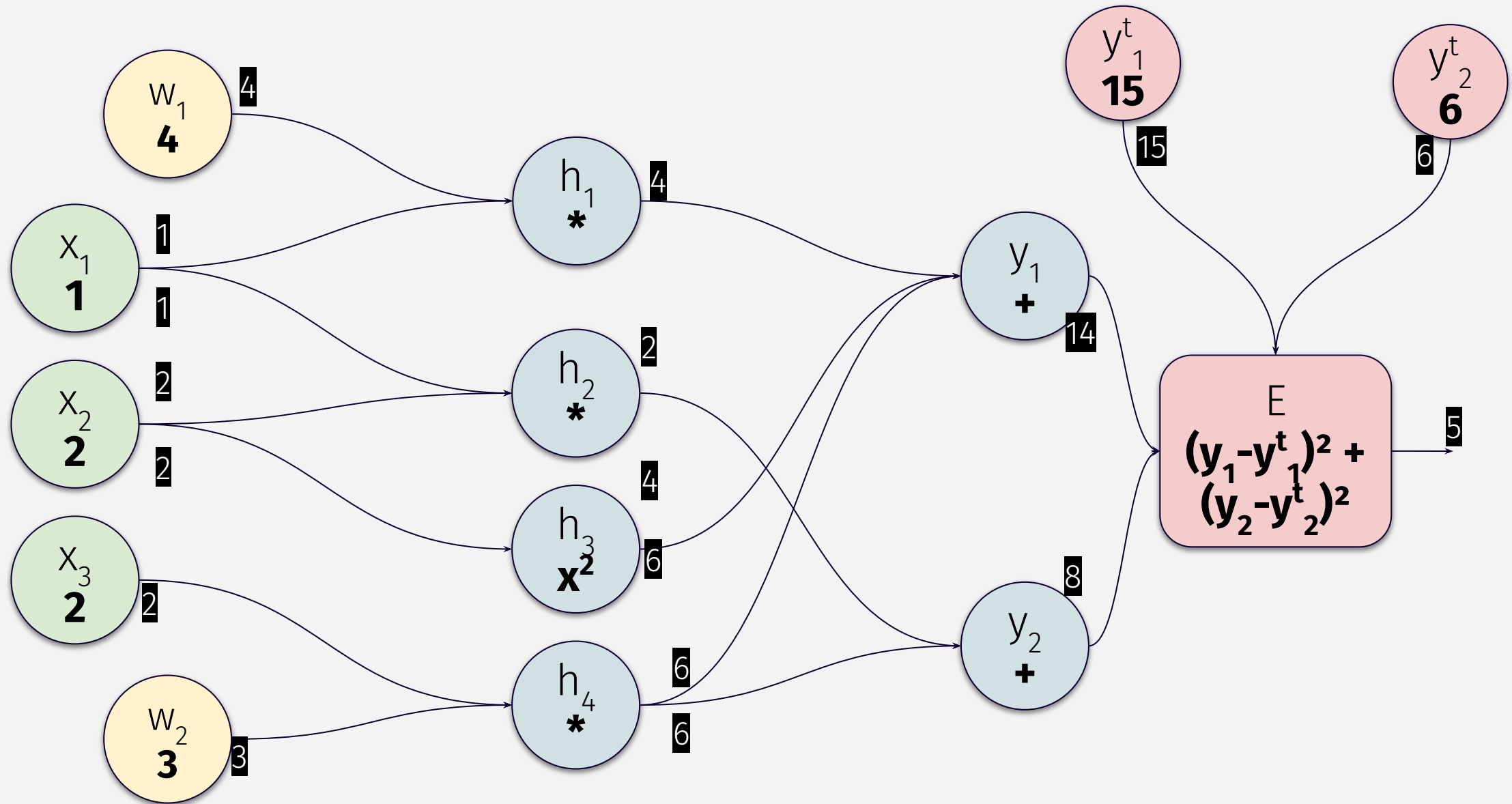


Recursos

- Visor interactivo de sigmoidea
- Visor interactivo de selector de región y asignador de valor con 2 sigmoideas
- Explicación más detallada (Lorke et al)
- Explicación con visualizaciones interactivas (Nielsen)

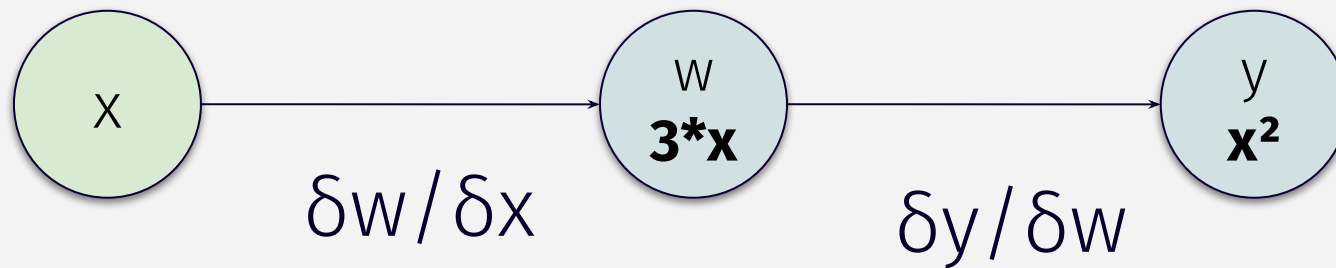
Backpropagation en Grafos Generales

Backpropagation en un Grafo - Forward



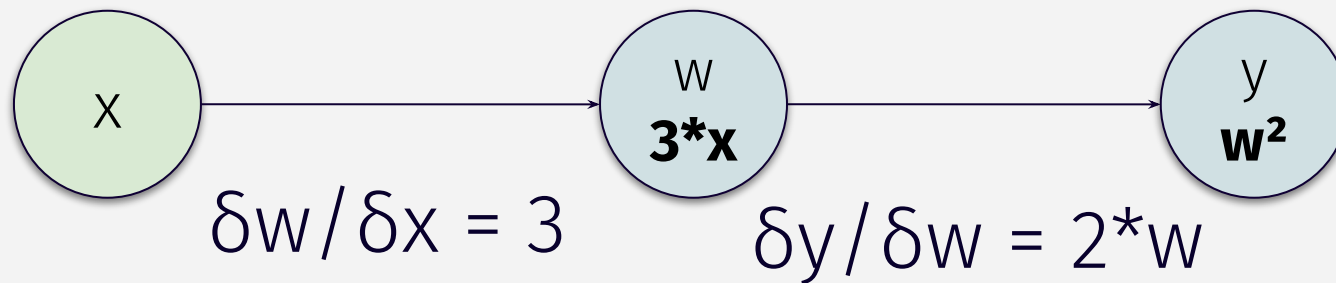
Regla de la cadena

- $f(g(x))' = f'(g(x)) * g'(x)$
- $\delta f(g(x)) / \delta x = \delta f(g(x)) / \delta g(x) * \delta g(x) / \delta x$
- Variables intermedias
 - $y = f(w)$
 - $w = g(x)$
 - $\delta y / \delta x = \delta y / \delta w * \delta w / \delta x$



Regla de la cadena - Ejemplo

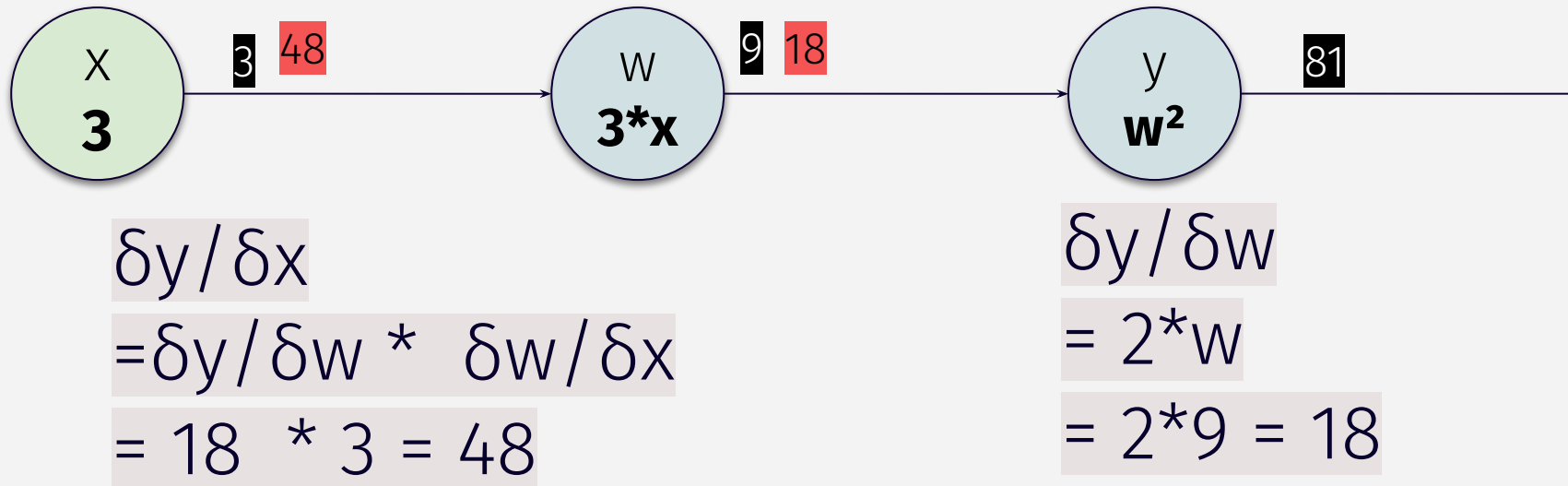
- $f(x) = x^2$
- $g(x) = 3 * x$
 - $f(g(x)) = (x * 3)^2$
- Variables intermedias
 - $y = f(w) = w^2$
 - $w = g(x) = 3 * x$



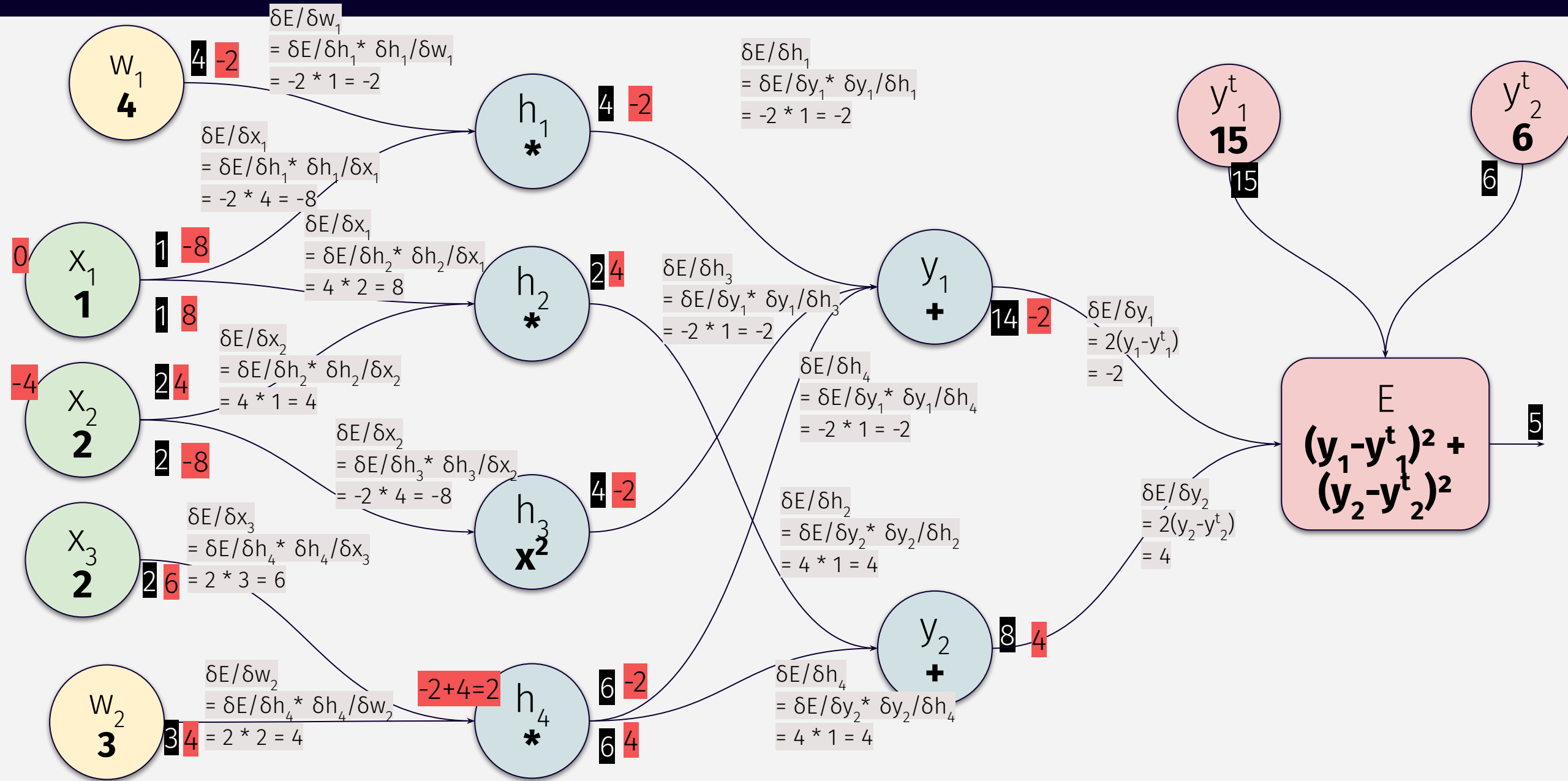
$$\delta y / \delta x = \mathbf{\delta y / \delta w * \delta w / \delta x} = 3 * (2 * w) = \mathbf{3 * (2 * (3 * x))} = 3 * 2 * 3 * x = \mathbf{18 * x}$$

Regla de la cadena - Ejemplo $x = 3$

- Forma analítica:
- $\delta y / \delta x = 18 * x$
 - $x = 3 \rightarrow \delta y / \delta x = 18 * 3 = 48$
- En el grafo:

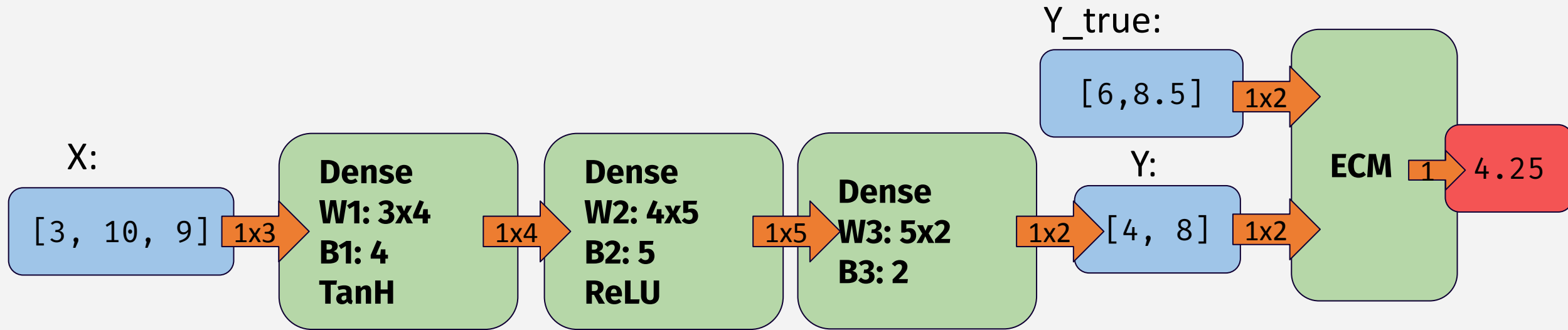


Backpropagation en un Grafo - Backward



Backpropagation en Redes Secuenciales

Backpropagation - Capas



- **Forward**

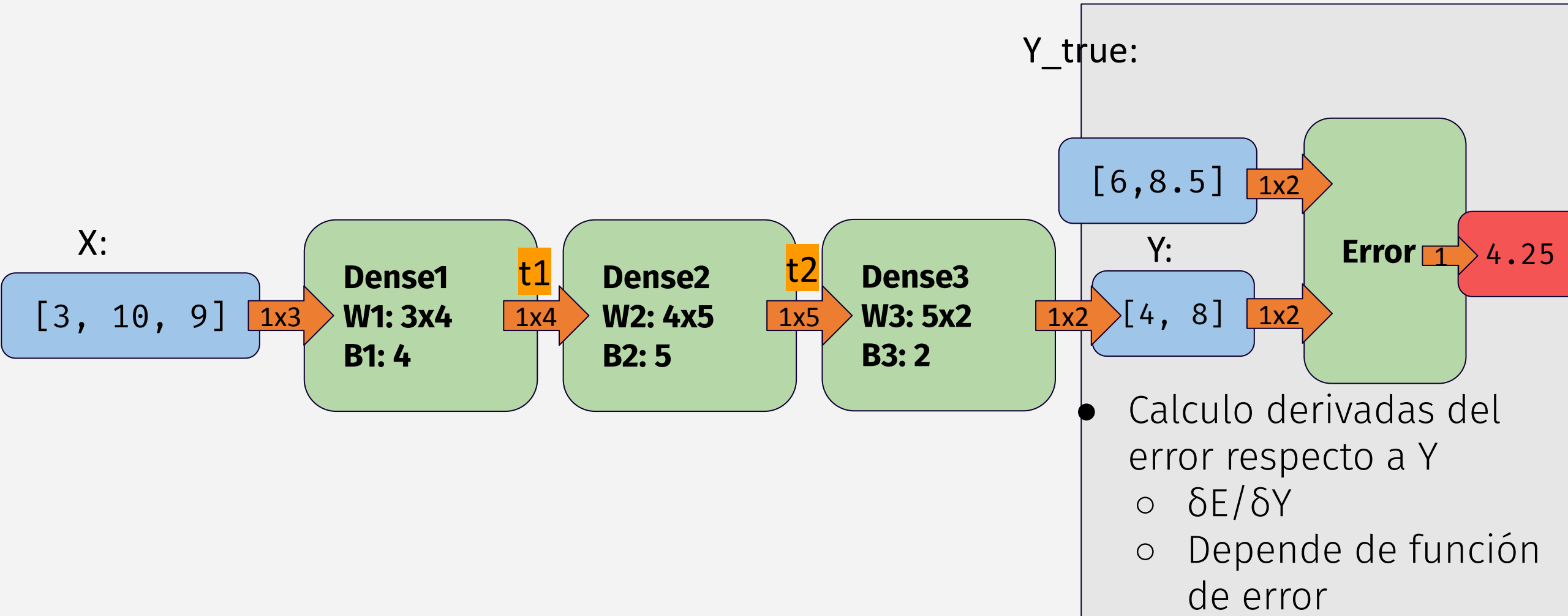
- De entrada a salida
- *Almacenar* valores intermedios

- **Backward**

- Derivadas de salida a entrada
- *Utilizar* valores intermedios

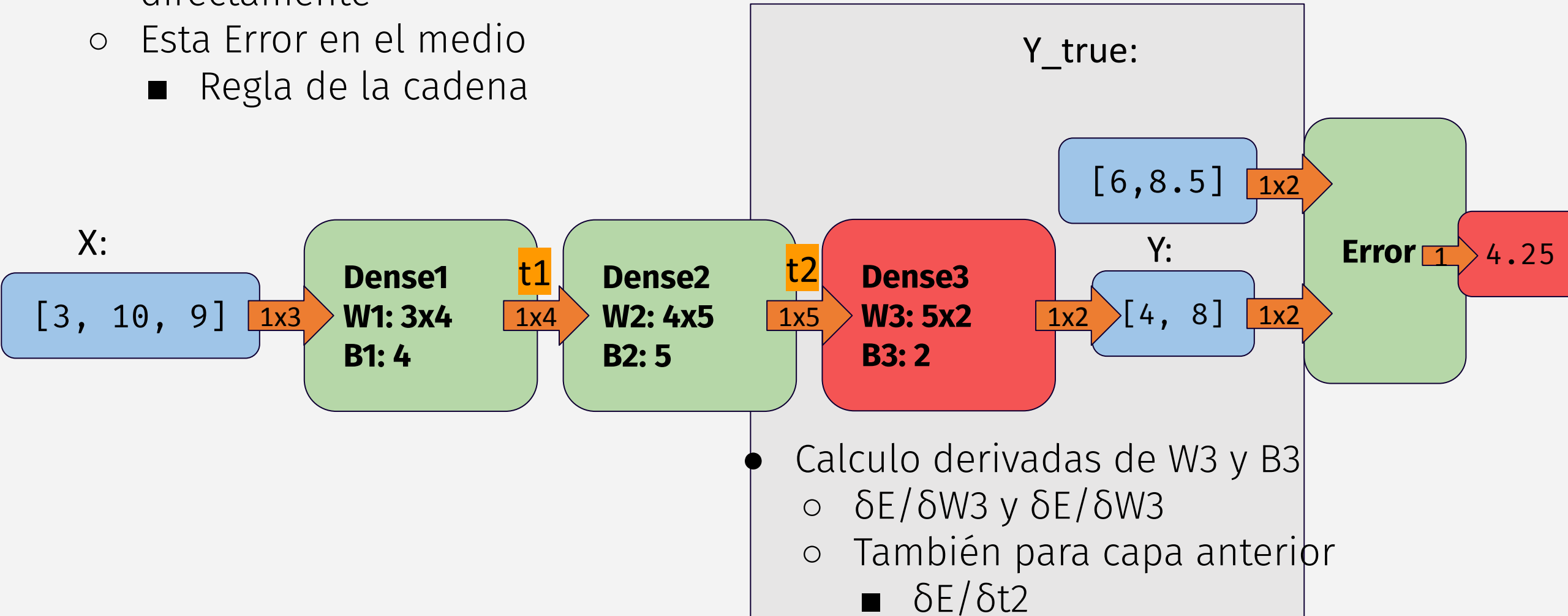
backward: Cálculo de derivadas por capas

- Derivada del error
 - Se calcula directamente



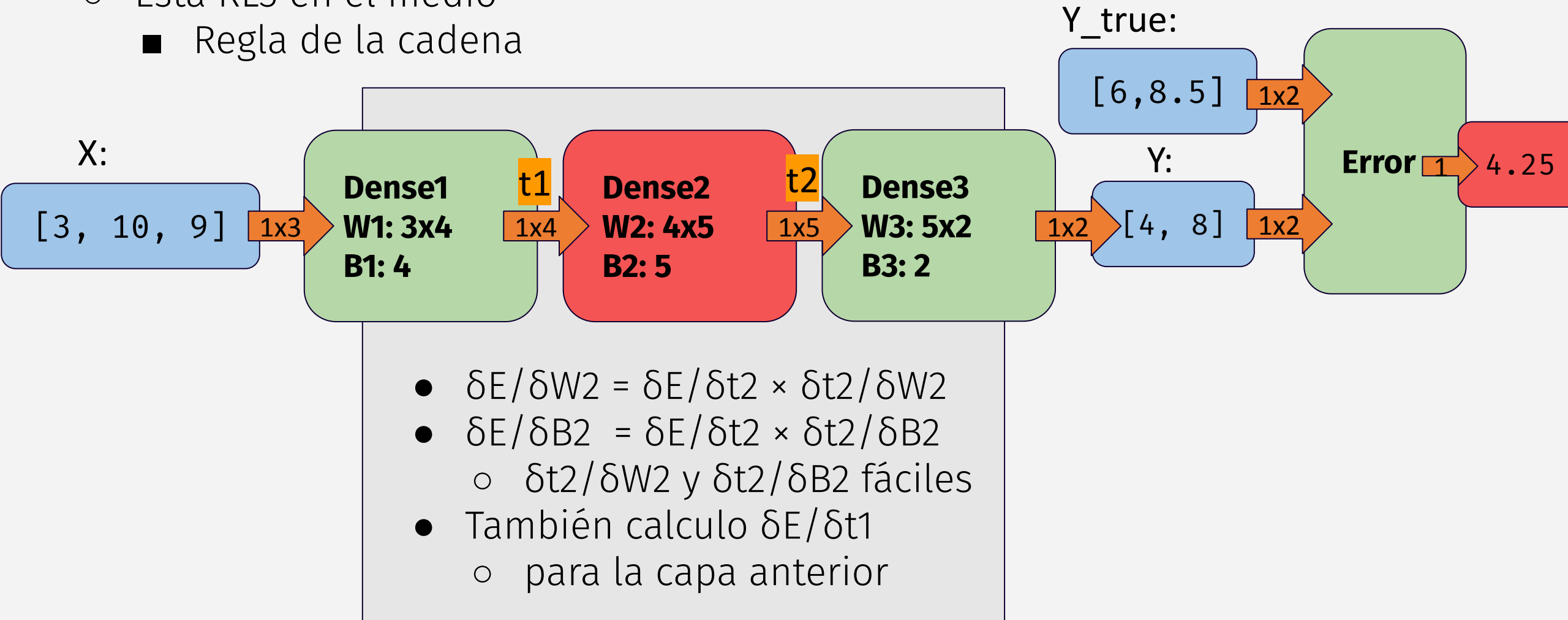
backward: Cálculo de derivadas por capas

- $\delta E / \delta W_3$ y $\delta E / \delta B_3$
 - No se pueden calcular directamente
 - Esta Error en el medio
 - Regla de la cadena



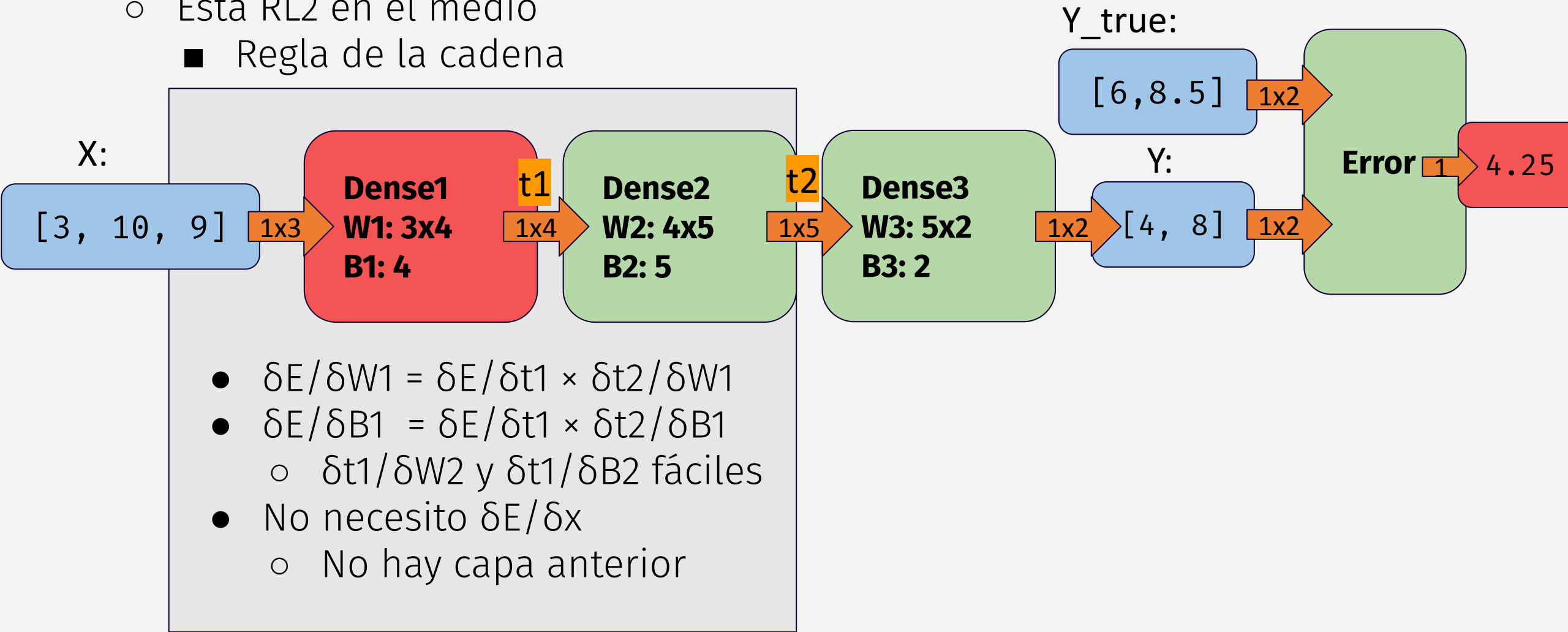
backward: Cálculo de derivadas por capas

- $\delta E / \delta W2$ y $\delta E / \delta B2$
 - No se pueden calcular directamente
 - Esta RL3 en el medio
 - Regla de la cadena



backward: Cálculo de derivadas por capas

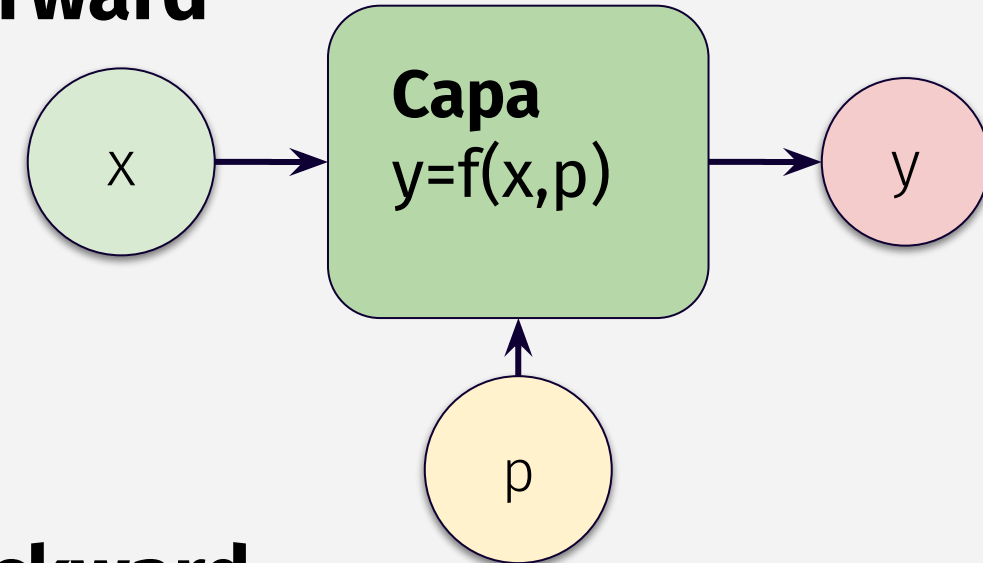
- $\delta E / \delta W1$ y $\delta E / \delta B1$
 - No se pueden calcular directamente
 - Esta RL2 en el medio
 - Regla de la cadena



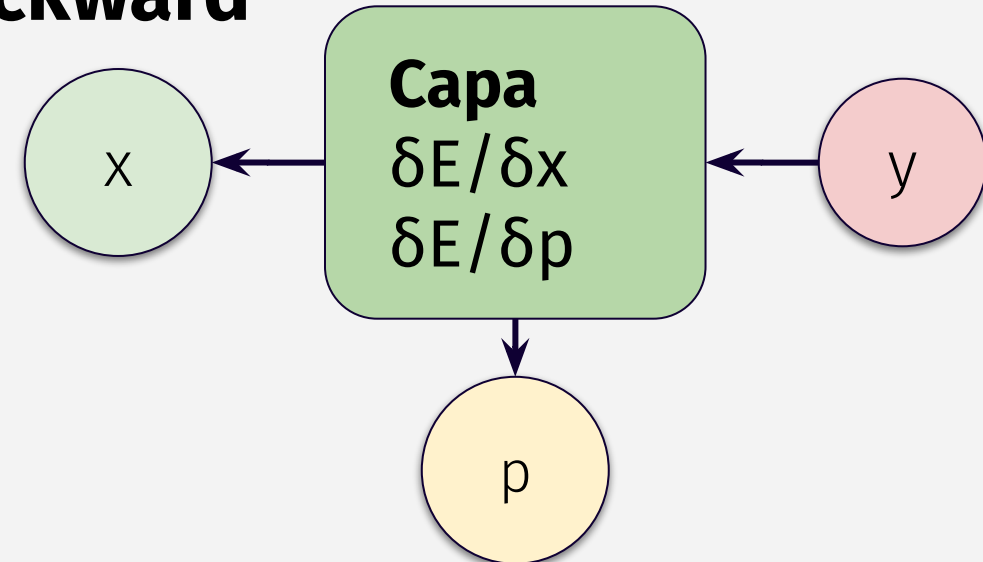
Cálculo modular por Capas

- Cada capa debe saber calcular
 - Forward: $y = f(x)$
 - Backward
 - $\delta E / \delta x = \delta E / \delta y * \delta E_y / \delta x$
 - $\delta E / \delta p = \delta E / \delta y * \delta E_y / \delta p$
- Modularidad
 - Capas independientes
 - Puedo intercambiar

Forward

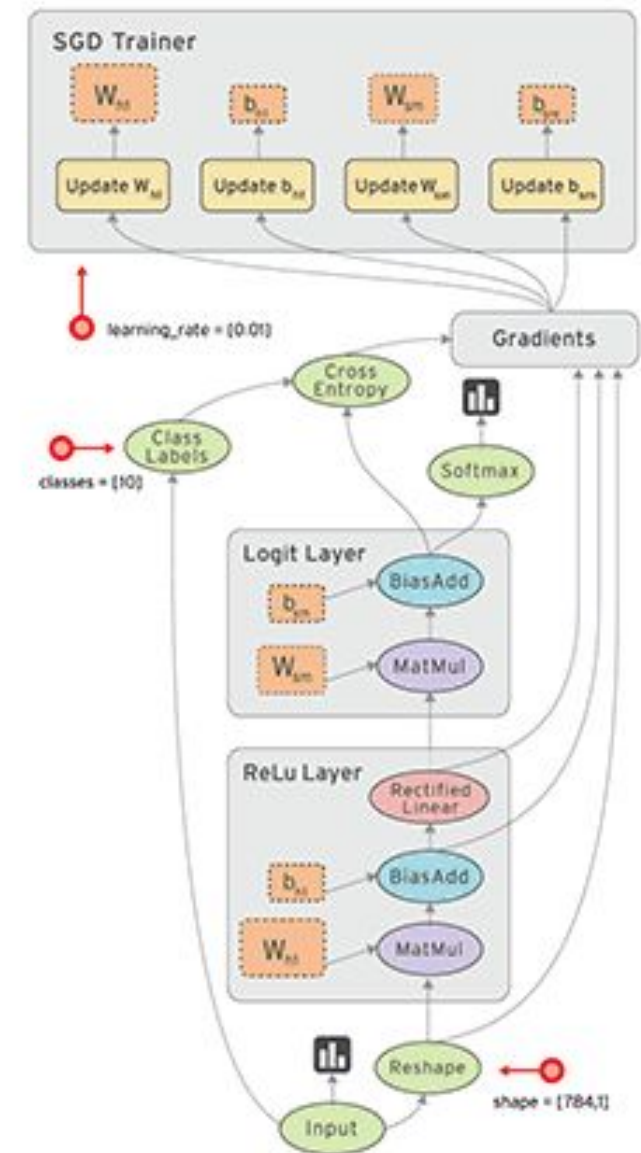


Backward



Descenso de gradiente en redes

- Descenso de gradiente
 - Igual que antes
 - Más parámetros
 - Misma idea
 - $w = w - \alpha \delta E / \delta w$
- Cálculo de derivadas
 - Backpropagation
 - Frameworks actuales
 - Especificás el **forward**
 - Capas/arquitectura de la red
 - El framework determina el backward
 - Backpropagation automático
 - Funciona por lotes



Modelos pre-entrenados, Transferencia de Aprendizaje y Finetuning

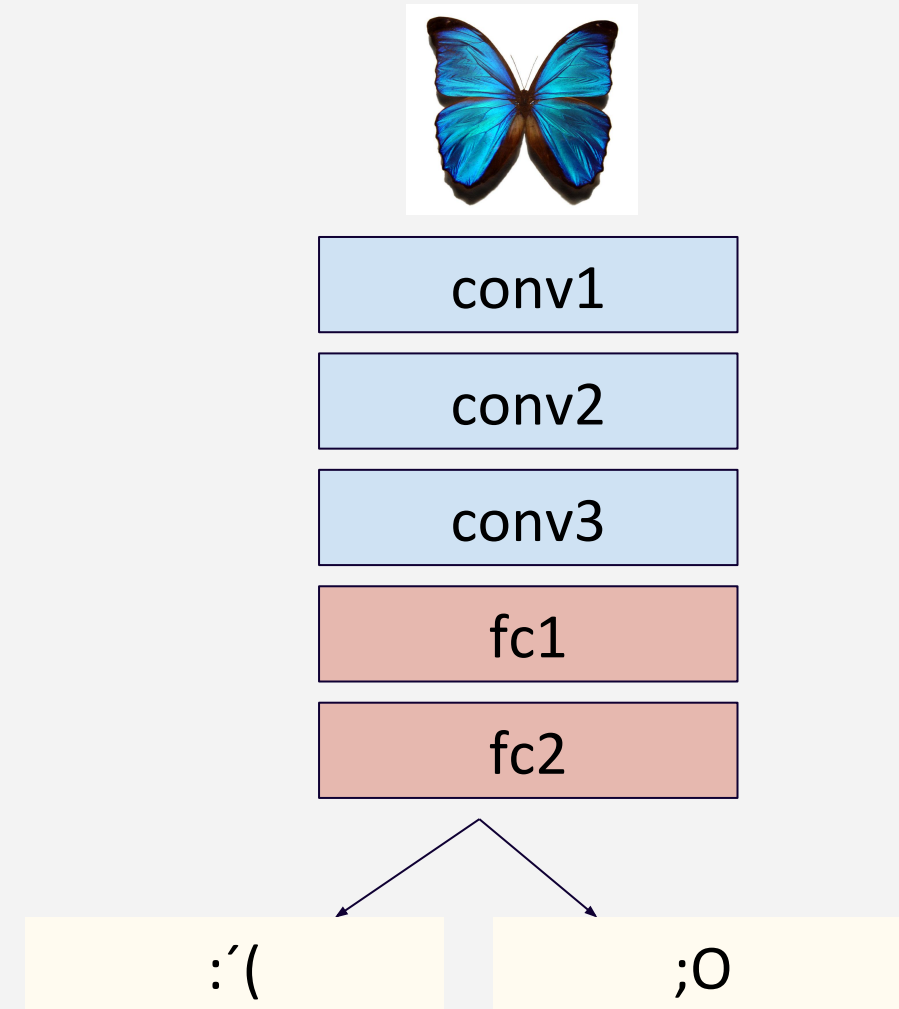
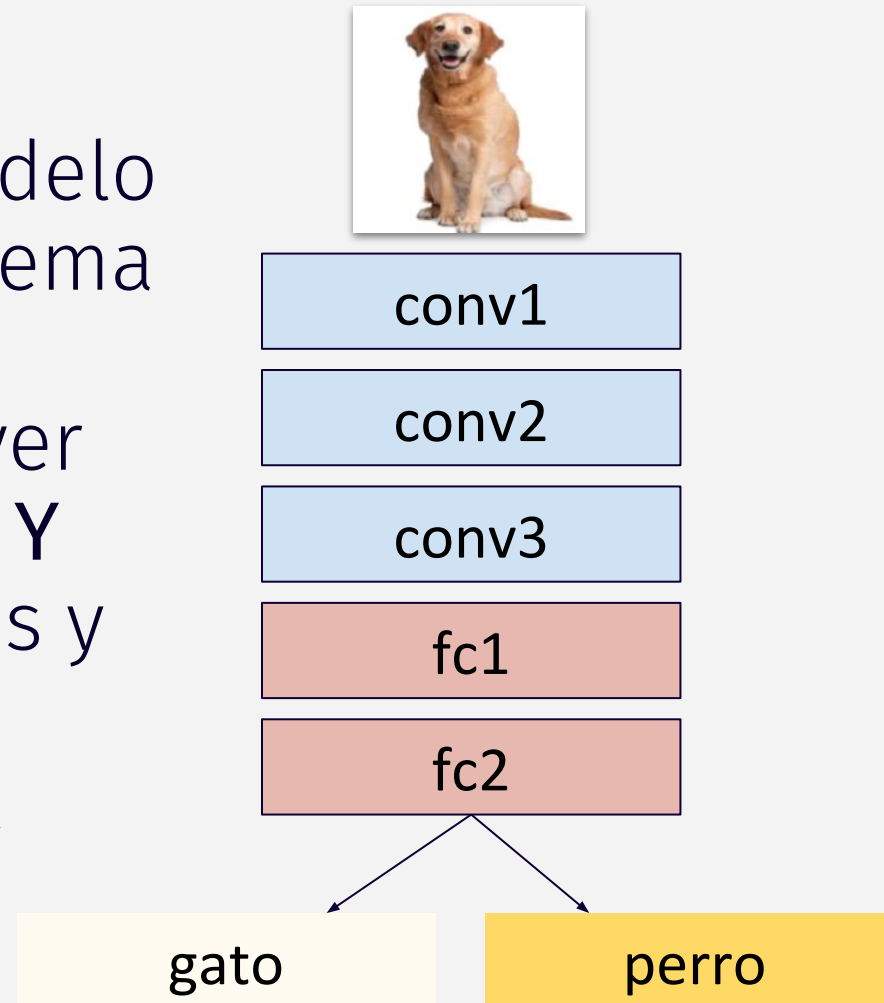
Modelos Preentrenados

- Entrenar un modelo requiere
 - Tiempo/uso de cpu
 - Optimización de hiperparámetros
 - Preparación de datos
- Usar un modelo preentrenado
 - + Fácil y rápido
 - - Arquitectura fija
 - - Dominio fijo

```
from keras.applications.mobilenet import
MobileNet, preprocess_input
model = MobileNet(weights='imagenet')
from keras.preprocessing import image
img = image.load_img(image_path, target_size=(224, 224))
img = image.img_to_array(img)
img = np.expand_dims(img, axis=0)
img = preprocess_input(img)
class=model.predict_classes(img)
```

Transferencia de Aprendizaje

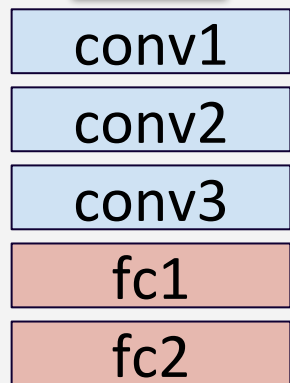
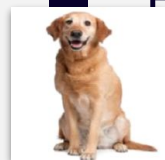
- Motivación
 - Tengo un modelo
 - para problema X
 - Quiero resolver
 - problema Y
- Modelo de Gatos y Perros
 - No sirve para Mariposas vs Abejas



Transferencia de Aprendizaje

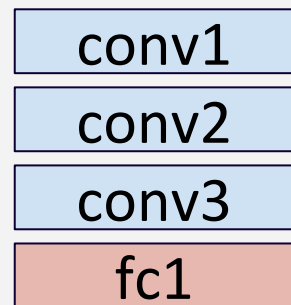
- ¿Empiezo desde 0? No
 - **Reusar Filtros Convolucionales**
 - Relativamente independientes del dominio
 - Reemplazar última capa Dense o GlobalAveragePooling
 - Reentrenar red
- **Congelar** capas anteriores

- **Congelar**
 - No entrenar los pesos

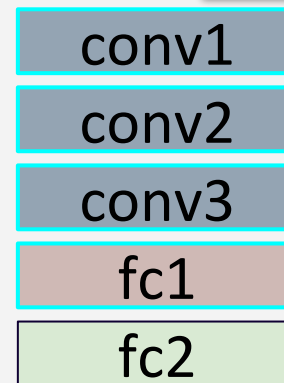
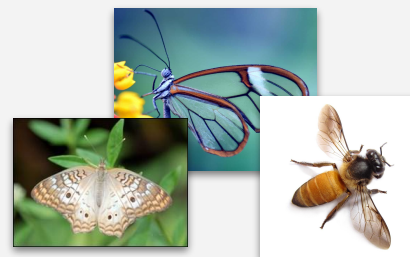


gato perro

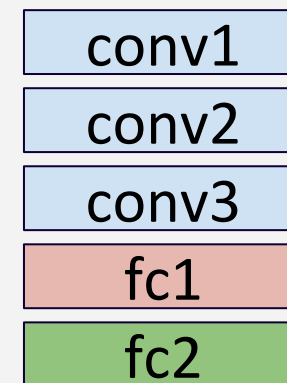
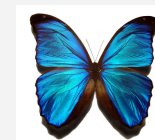
Quitar fc2



Agregar fc2



Reentrenar



Mariposa Abeja

Transferencia de Aprendizaje ([kaggle](#),[notebook](#))

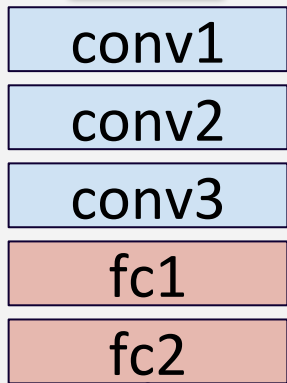
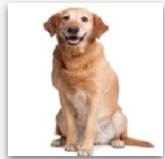
```
base_model=MobileNet(input_shape=(32,32,3),weights='imagenet',include_top=False)
for layer in base_model.layers:
    layer.trainable=False # capas “congeladas” no se entrenan

# Utilizar salida del modelo como entrada a capa Dense de 128
output = Dense(128,activation='relu')(base_model.output)
# Nueva capa de salida
output = Dense(classes,activation='softmax')(output)
model=Model(inputs=base_model.input,outputs=output)

#Entrenar con nuevos datos
model.compile(optimizer='Adam',loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(x,y,epochs=20)
```

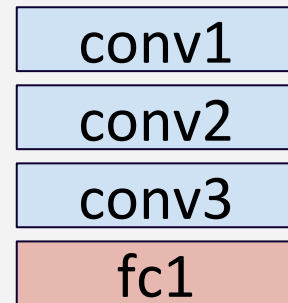
Finetuning

- Reentrenar todo el modelo
 - Más tiempo de entrenamiento
 - Resultados ligeramente mejores (1% a 5%)
- Código: Igual que el anterior
 - Pero sin congelar pesos

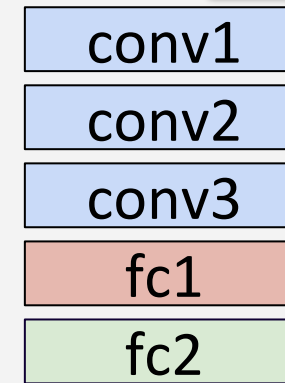
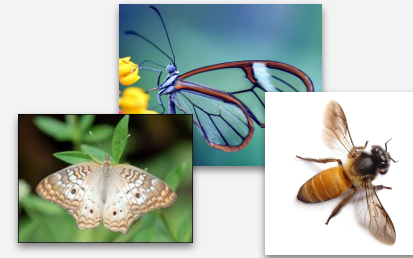


gato perro

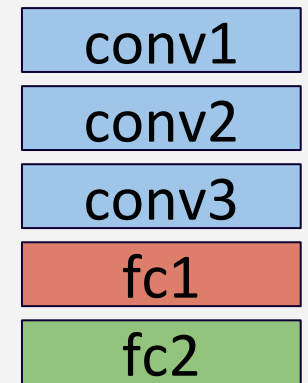
Quitar fc2



Agregar fc2



Reentrenar



Mariposa Abeja